

OS fogalom, felépítése

Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek
jegyzete alapján készült el

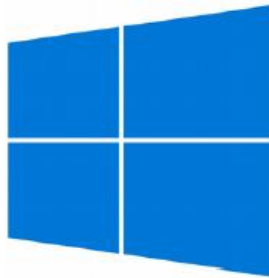
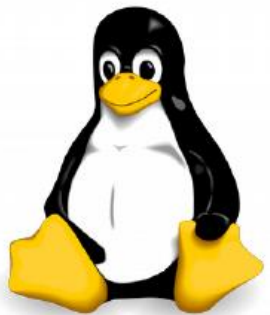
Témakörök

- Az OS fogalma
- OS osztályozási szempontok
- OS működésének alapelvei
- Az OS-ek felépítése
- Implementációs struktúrák

OS fogalma - több nézőpont

„A témakör megközelíteni (Operating System)

Megvizsgáljuk az *alapvető feladatait* egy mai modern operációs rendszernek (pl.: Linux, Android, Windows, Mac OS, UNIX-ok, etc.)



OS fogalma - több nézőpont

Felhasználóként már találkoztak ezekkel (OS) a feladatokkal.

De nem gondolták, nem is tudták, hogy azokat a funkciókat valójában az....

operációs rendszer végzi el.

Melyek ezek a feladatok...

OS fogalma - több fogalom

1. *Erőforrás menedzser (Resource Manager)*

- Hatékonyá, biztonságossá, igazságossá teszi a géphasználatot.

2. *Kiterjesztett gép (Extended Machine)*

- Kényelmessé, "felhasználóbarát" jellegűvé teszi a géphasználatot.

3. *ISO szabvány szerinti* (nemzetközi szabványosítási szervezet definíciója)

OS fogalma - erőforrások menedzser

Erőforrások menedzselése

- *Erőforrások:*

- hardver (processzor, tárák stb.)
- szoftver (alkalmazások, adatbázisok stb.)
- emberi (felhasználók, rendszergazdák stb.)

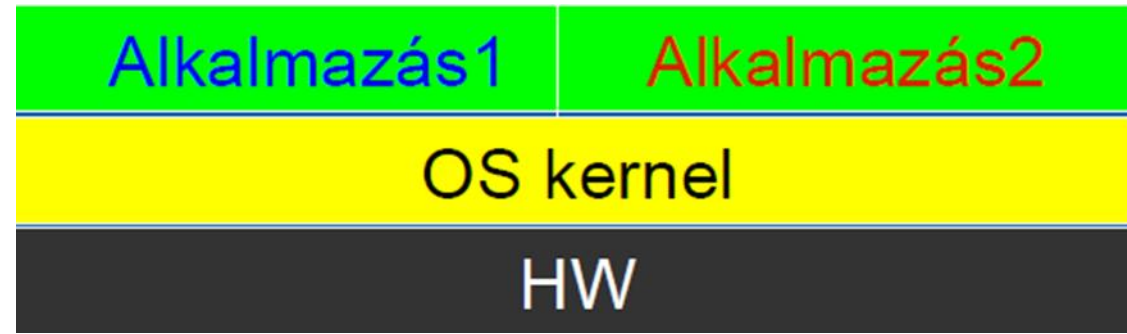
Def: Minden, ami egy processz végrehajtásához szükséges.

OS fogalma - erőforrások menedzser

- *Menedzselés:*

- *Elosztja az erőforrásokat* a processzek között (diszkek, perifériák, tárak, processzorok)
- *Védelem* (kölcsonös kizárás – a *kritikus szakaszok*)
- *Nyilvántartás* (statisztikát készít az erőforrások használatáról)

OS fogalma – hatékonyság



Hatékonyság, biztonságossá, igazságossá teszi a géphasználatot - azért mert...

*Az OS feladatát **alulról**, a hardver oldaláról fogalmazzuk meg.*

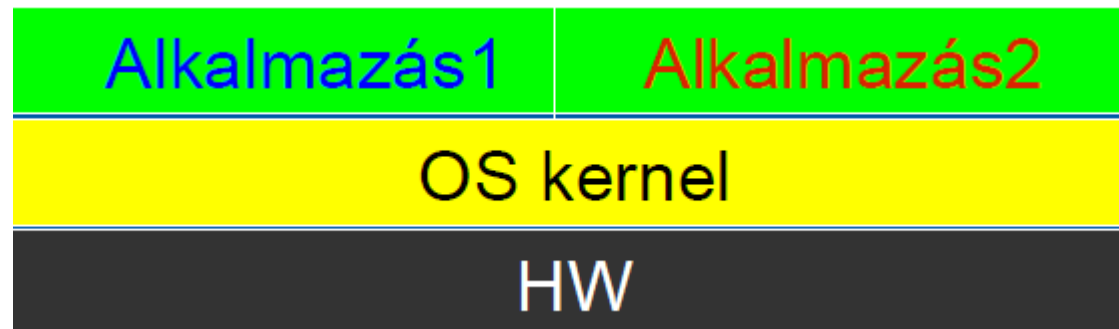
A felhasználók elsődleges célja az erőforrások elérése, azaz használni szeretnék.

Így sokszor az OS-nek kell dönteni, hogy igényelt erőforrásokat melyik felhasználóhoz rendeli.

OS fogalma – kiterjesztett gép

Kiterjesztett gép (Extended Machine)

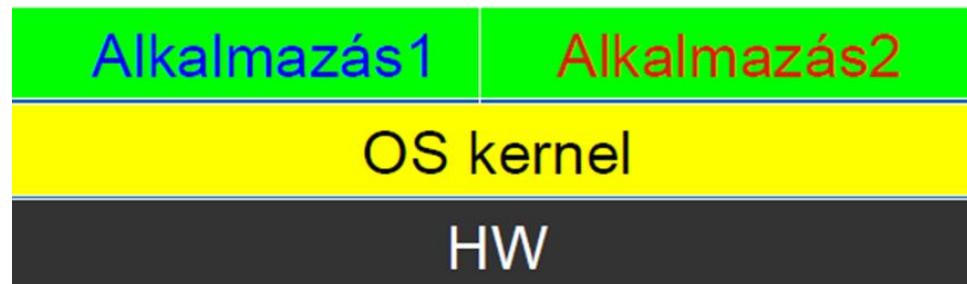
- *Magasabb absztrakciós szintet biztosít a felhasználó számára* (szimbolikus nevek a fájlok számára)
- *Egységességet kínál a hasonló eszközök kezeléséhez.*
- *Helyettesíthetőséget biztosít.*



OS fogalma – kiterjesztett gép

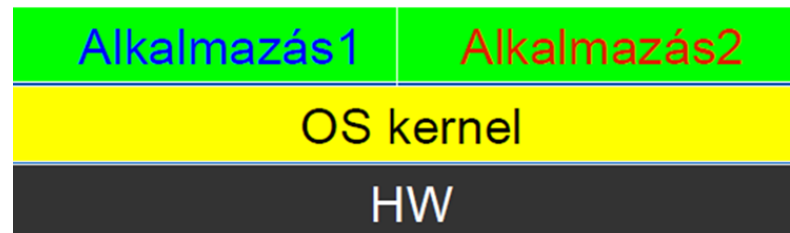
A legtöbb számítógép architektúrája a gépi nyelvű szintjén való programozása kényelmetlen.

- A programozó nem kíván belemerülni egy-egy hardvereszköz programozásába.
- Ehelyett a programozó egy *magas szintű absztrakciós szinten* szeretne foglalkozni.



OS fogalma – kiterjesztett gép

- Ez azt jelenti, hogy lényeges és lényegtelen tulajdonságtól elválasztani, a *lényeges tulajdonságok* kiemelni és a *lényegtelen tulajdonságok* figyelmen kívül hagyni.
- Az a *program*, amely elrejtí a *hardvert* a programozó elől, az *operációs rendszer*.
- Ebből a nézőpontból az operációs rendszer olyan *kiterjesztett gép*, amelyet *egyszerűbb programozni*, mint a *mögötte lévő hardvert*.

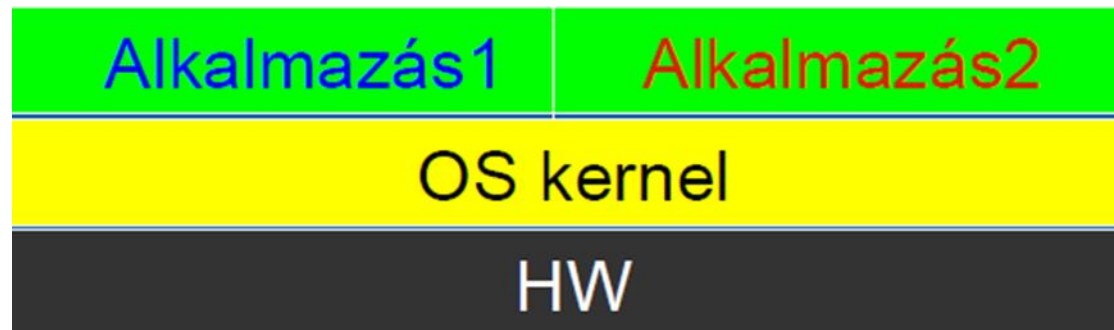


OS fogalma – kiterjesztett gép

- Magas szintű operációk (*open, read, write, system calls, etc.*), amely az OS-hez kötődnek,
- Parancsértelmezők - magasabb szintű parancsokat (*copy, cut, paste, move, etc.*)

OS fogalma – kényelmes használat

- A *kényelmes* használat azt jelenti, hogy „**felülről**”, a felhasználó oldaláról közelítünk az operációs rendszerhez.
- Az OS egy *réteget képez a hardver fölött, kibővíti a számítógép hardver szolgáltatásait.*
- A felhasználó egy *kényelmesebb tulajdonságokkal* rendelkező **virtuális gépet** lát.



OS fogalma - ISO

ISO (Erőforrás menedzselés szempont)

„Olyan *programrendszer*, amely a számítógépes rendszerben a programok végrehajtását vezérli:

- ütemezi a processzek végrehajtását,
- elosztja az erőforrásokat,
- biztosítja a felhasználó és a számítógépes rendszer közötti kommunikációt.”

OS osztályozási szempontok - hardver mérete

1. Az operációs rendszer alatti *hardver "mérete"* szerinti osztályozás szerint:

- *mikroszámítógépek* operációs rendszerei,
- *kisszámítógépek* (esetleg munkaállomások) operációs rendszerei,
- *nagygépek* (Main Frame Computers, Super Computers) operációs rendszerei.

OS osztályozási szempontok - kapcsolattartás típusa

A kapcsolattartás típusa szerinti osztályozás:

- *köteget feldolgozású* operációs rendszerek, vezérlőkártyás kapcsolattartással.
- *interaktív operációs* rendszerek.

OS osztályozási szempontok – Cél

2. OS cél szerint:

- *Általános célú OS-k*: egyidejűleg több célra is használhatók – pl.: program-fejlesztésre, alkalmazások futtatására, adatbázisok lekérdezésére, kommunikációra.
- *Speciális célú rendszerek OS-k*: csak egy bizonyos célt szolgálnak pl.: *folyamatvezérlésre* beállított, *tranzakció feldolgozásra* implementált stb. rendszerek.

OS osztályozási szempontok – Processz kezelés

3. *Processz kezelés, CPU időkiosztás, felhasználó szám szerinti osztályok*

a.) *Single processing: egy processzoros gépek működtető rendszere:*

- *single tasking* (egyenként egy processz lehetséges, azaz egyszerre csak egy feladattal foglalkozhat),
- *multi tasking* rendszer (párhuzamosan több processz is futhat, így egyidőben több feladatot is kezelhet).

OS osztályozási szempontok – Processz kezelés

b.) *Multi processing*: több processzorral rendelkező gépek és ezek OS-ei.

c.) *Egyidejű felhasználók száma* szerint:

- egyfelhasználós (single user) rendszerekről és
- többfelhasználós (multi user rendszerekről.)

OS osztályozási szempontok – CPU időkiosztása

d.) A CPU időkiosztása:

- *Szekvenciális*: egy processz teljes feldolgozása után kapcsol a másik processzre,
- *Kooperatív-event polling rendszerű*: a processzek egy előre beállított körkörös sorrendet követnek, s az a processz lesz aktív, amelyik eseményt (billentyű lenyomás) kap.

A kooperatív rendszernél egy-egy processz saját elhatározásából is lemondhat a processzorról és átadja a vezérlést a másik processznek.

OS osztályozási szempontok – CPU időkiosztása

- *Megszakításvezérelt* (interrupt driven): minden *I/O megszakítás* esetén *újraértékelik* a processzek *prioritási állapotait*, és a legmagasabb prioritású processz kapja a CPU-t.
- *Beavatkozó-megszakításvezérelt* (preemptive interrupt-driven): nemcsak az *I/O megszakításoknál* értékelik újra a *prioritási állapotokat*, hanem bizonyos *óra megszakításoknál* is.

OS osztályozási szempontok - Példák

- **MS DOS** általános célú, egyfelhasználós, egyfeladatos, időosztású operációs rendszer.
- **MS Windows:** általános célú, egyfelhasználós, többfeladatos rendszer.
- **OS/2:** általános célú, egyfelhasználós, többfeladatos rendszer.

Igazi operációs rendszer: *időkiosztásos, beavatkozó-megszakítás-vezérelt.*

OS osztályozási szempontok - Példák

- **Windows NT:** általános célú, többfelhasználós, többfeladatos rendszer (multi-processing).
- **Win 95:** egyprocesszoros, általános célú, Intel platformú, többfeladatos, beavatkozó-megszakításvezérelt.
- **Unix rendszerek:** általános célú, többfelhasználós, többfeladatos rendszerek.

OS osztályozási szempontok- Memória menedzselés

e.) Memória menedzselés szerinti osztályozás:

Alapvetően megkülönböztetünk *valós címzésű* és *virtuális címzésű* rendszereket:

- *valós címzésű rendszereken* belül a *fix*, vagy *változó partíciókra* osztott memóriakezelés,

OS osztályozási szempontok- Memória menedzselés

- *virtuális címzésű rendszerek* esetén:
 - ki/be söprő (swapping in/out) rendszerek,
 - ki/be lapozó (demand paging) rendszerek,
 - ki/be söprő és lapozó (swapping and paging) rendszerek.

OS osztályozási szempontok - Példák

- **MS DOS:** valós címzésű, változó partíciókra osztó rendszer.
CP/M (Control Program for Microcomputers, Z80), AmigaOS (Motorola 68000), IBM Mainframe rendszerek (System/360)
- **Windows NT:** virtuális címzésű, lapozó rendszer.
- **UNIX:** virtuális címzésű rendszer.

A korai Unix-ok ki/be söprő rendszerek, ma már ki/be söprő és lapozó a memóriakezelés.

OS osztályozási szempontok- I/O koncepciók

f) *I/O koncepciók, a fájlrendszer megvalósítása szerinti osztályozás.*

Sokféle megoldást találunk az egyes operációs rendszerek implementációiban.

- Pl.: vannak OS-k, amelyek egy sajátos fájlrendszert képesek kezelni (pl. az MS-DOS a FAT-os fájlrendszert,
- Vannak OS-k, melyekhez tartozik ugyan saját fájlrendszer, de más koncepciójú fájlrendszereket is kezelnek (pl. a Linux meglehetősen sok fájlrendszert ismer)

OS osztályozási szempontok - Fájlrendszerek

A mai operációs rendszerek egyszerre több fajta *fájlrendszer kezelését* is el tudják végezni. Elterjedt fájlrendszer: Pl.

- **FAT**, ami a DOS operációs rendszerek fájlrendszere is volt, de manapság is használják pl.: pendrive, memóriakártya - az egyszerű szerkezete miatt.
- **NTFS**, ami Windows operációs rendszereken elterjedt.
- **ext2, ext3, ext4** (extended file system), ami Linux operációs rendszereken jellemző.

OS osztályozási szempontok - Fájlrendszerek

Speciális fájlrendszer típusok, pl.

XFS, HPFS, JFS, UFS, stb.

Az optikai lemezek (CD/DVD/Blu-ray) többnyire az ISO9660 szabványos fájlrendszert, vagy az UDF (Universal Disk Format, Univerzális lemezformátum) fájlrendszert használják.

OS osztályozási szempontok - Fájrendszer

- *ReiserFS*: naplózó, jobb lemezterület kihasználás, megbízható.
- *XFS (Extended)*: valós idejű defragmentálás, 64 bites, naplózó.
- *JFS: (Naplózott)*: Nagyon gyors, notebookba ezért jó, 64 bites.
- *Btrfs*: B-tree, Oracle, új sok funkció, - következő Linuxos fájlrendszer”.
- *ZFS: (zettabyte)* Sun Microsystems, 128 bites fájlrendszer
Támogatott: tömörítés, titkosítás.

OS osztályozási szempontok - I/O koncepciók

I/O koncepciók, fájlrendszer alapján

- A blokkok hozzárendelése a fájlnevhez - hogyan történik?
 - Folyamatos allokációval (már nem használatos).
 - Indextáblás módszerrel (UNIX is ilyen).
 - láncolt listás allokáció index táblával,
 - i-node módszer.

OS osztályozási szempontok- I/O koncepciók

- Hogyan kezeli az OS a *szabad blokkokat*?
 - *Bit-térképekkel/foglaltsági térképekkel* (lehet egyetlen az eszközön, de lehet megosztott, több darabból álló is)
 - *Láncolt listában tárolja.*

OS osztályozási szempontok - összefoglalás

- *Hardver függőség szerint:*
 - Személyi, kis, nagy gépek OS-e, architektúra független.
- *Cél szerint*
 - Általános, vagy speciális célú.
- *Processzorok, processzek, felhasználók száma szerint:*
 - multi processing,
 - single, vagy multi tasking,
 - single, vagy multi user.

OS osztályozási szempontok - összefoglalás

- *Időosztás szerint*
 - Szekvenciális, kooperatív, vagy beavatkozó.
- *Memóriakezelés szerint*
 - Valós címzésű, vagy virtuális címzésű
- *I/O és fájlrendszer megvalósítás szerint etc.*

OS működése – ismétlés

Hogyan működik a számítógép: a CPU veszi a soron következő *instrukciót* és végrehajtja ...

- A CPU szemszögéből az *instrukciófolyam* - az *strukturálatlan*.
(nem érdekes számára melyik instrukció melyik programhoz tartozik)
- A *HW ad* egy strukturáltságot: *user módban/kernel módban végrehajtott instrukciófolyam* ún. *szakaszok* vannak.

OS működése – ismétlés

Logikailag szakaszoljuk az instrukciófolyamot.

- *Egy adott program futásához tartozó instrukciófolyam a kezdetétől a végéig - ez lesz egy szakasz.*
- *Ez a processz (job/taszk).*

Ezt a szakaszolást az OS végzi.

OS működésének alapelvei - Váltogatva futtatás

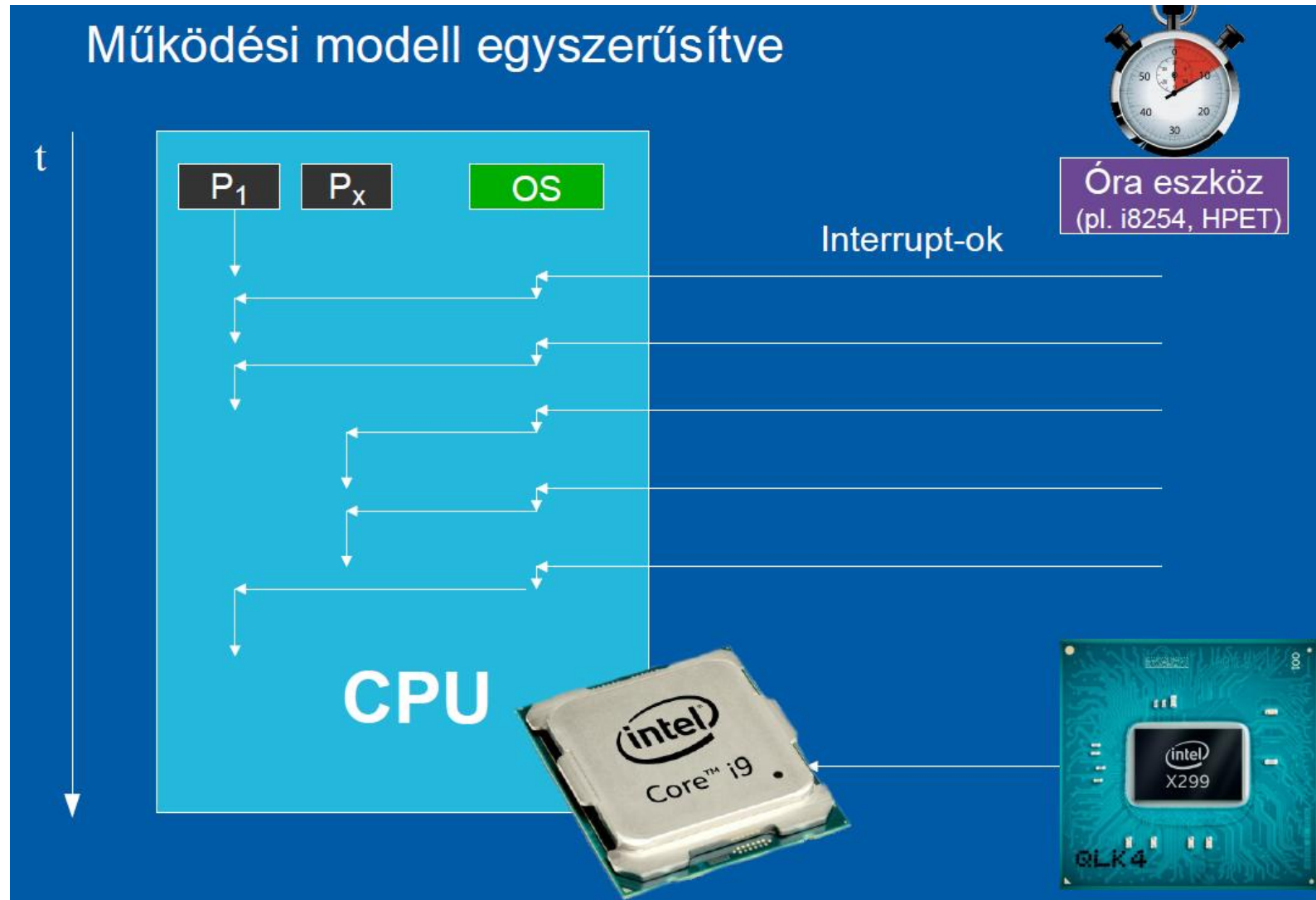
Néhány elv kiragadva:

1. *Váltogatva futtatás* megoldás:

több processz fut →

váltogatva a futtatás → *óra eszköz megszakítás, ütemező.*

OS működésének alapelvei - Váltakatva futtatás



OS működésének alapelvei – i8254

- i8254 *High Precision Event Timer* (HPET) - *nagy pontosságú eseményidőzítő* (HPET) *hardver* az Intel és a Microsoft 1. változatának specifikációit követi.
- HPET-nek van egy fix sebességű számlálója (10 MHz frekvencián, ezért „nagy pontosságú”) és akár 32 komparátor.
- Általában három vagy több komparátor áll rendelkezésre, amelyek mindegyike képes megszakításokat létrehozni.

További információk:

<https://www.kernel.org/doc/html/latest/timers/hpet.html>

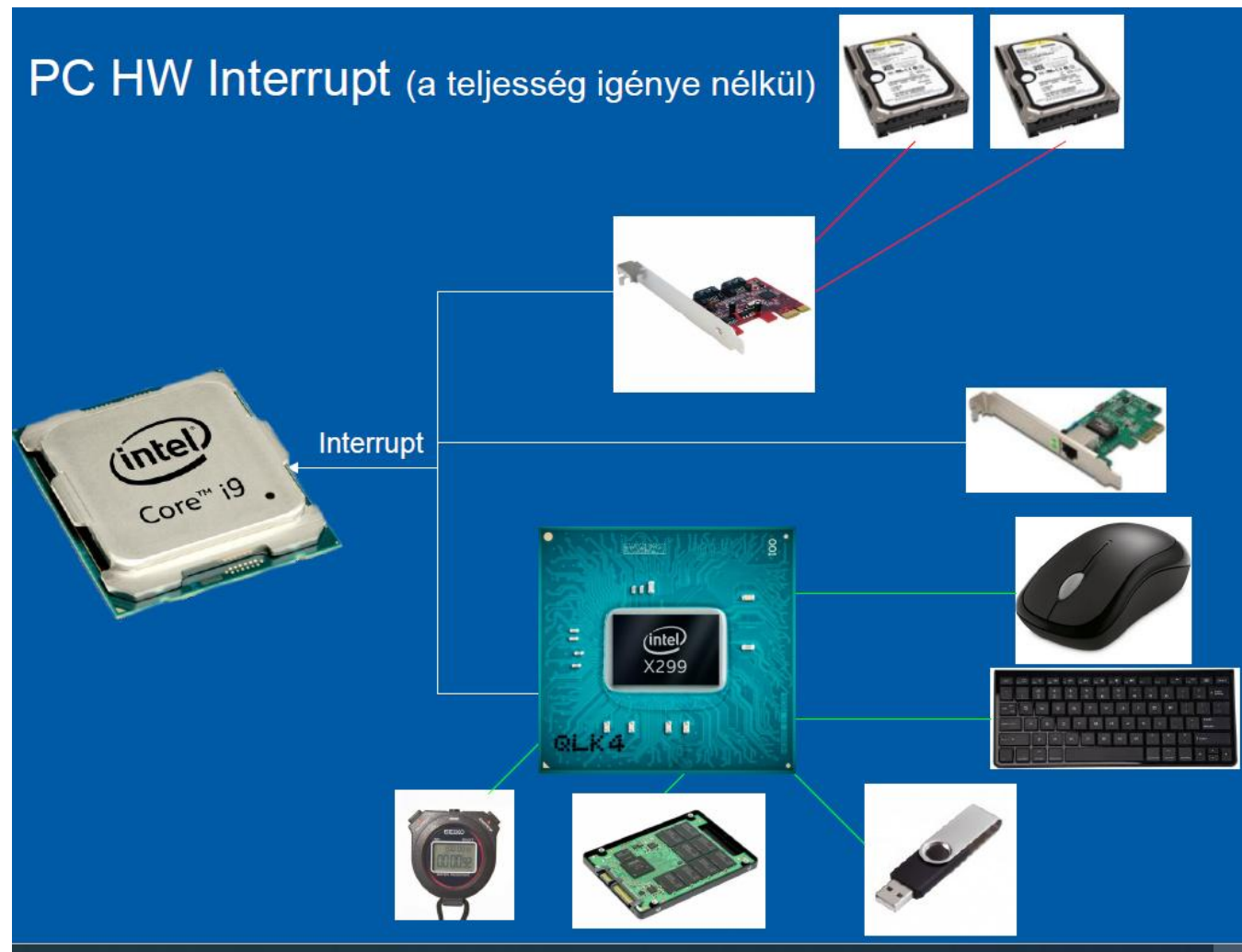
OS működésének alapelvei - Erőforrás kezelés

2. *Erőforrás kezelés* megoldása

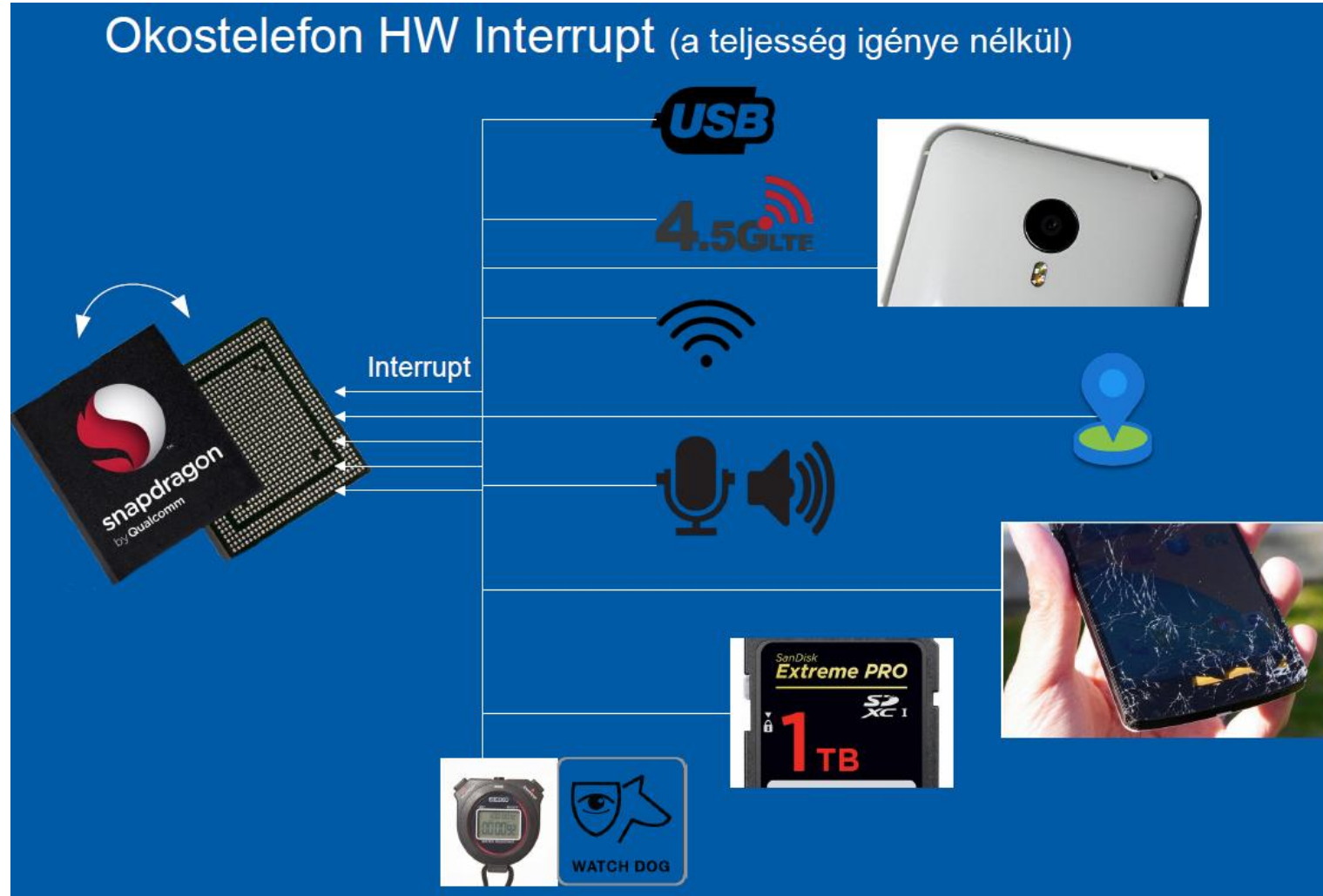
Az erőforrásokhoz hozzáférés csak az OS-en keresztül →
a processzek az *erőforrás kezeléseket* az OS-től kell kérjék:

System Call (rendszerhívás)

OS működésének alapelvei



OS működésének alapelvei



OS működésének alapelvei – OS vezérlés

Hogyan juthat a *vezérlés az OS-hez*?

- a.)** A felhasználói processzekből *rendszerhívással* (System Calls).
- b.)** Hardverből *megszakítással* (Interrupt).
- c.)** Hardverből *kivételes esemény* (Exception Condition) bekövetkezésével (Váratlan, de szinkron.)

Operációs rendszer felépítése

Operációs rendszer - mint *termék* tartalma.

- *Szűkebb értelemben* vett operációs rendszer tartalma:
 - *Kernel*: más nevén *rendszermag*, az operációs rendszer *funkcióit ténylegesen megvalósító* része.
 - *Egyéb részek*: felhasználói felület (burok, GUI), segédprogramok.

Operációs rendszer – kernel szolgáltatásai

Kernel struktúra (belső felépítés) több nézőpontból:

A.) Milyen szolgáltatásokat (funkciókat) biztosít?

(funkcionális struktúrák)

B.) Milyen felületeket (interface) ad a programozóknak, felhasználóknak?

C.) Implementációs struktúra. Komponenseit hogyan állítják elő, köztük milyen interfészek vannak?

Operációs rendszer – kernel szolgáltatásai

A.) Milyen *szolgáltatásokat* (funkciókat) biztosít?

(funkcionális struktúrák)

- *Processz* (taszk, fonal) menedzsment komponensek.
- *Memória* menedzselő komponensek.
- *I/O* menedzsment komponensek.
- *Fájlrendszer*t megvalósító komponensek.

(Komponens – összetevők)

Operációs rendszer – processz menedzsment

Processz (taszk, fonal) menedzsment komponensek funkciói:

a) Processzek *kreációját és terminálódását* biztosító funkciók.

b) Processz/task/fonal *kontroll, ütemezés:*

- állapotok nyilvántartása,
- CPU ütemezés, kiosztás, felfüggesztés, engedélyezés,

Operációs rendszer – processz menedzsment

- processz *szinkronizációs mechanizmusok* (beleértve a *holtpont elkerülését* biztosító és a *kölcsönös kizárási mechanizmusokat*),
- processz közti kommunikációs mechanizmusok (*pl. fájlokon, vágólapon, osztott memórián, semaforok segítségével komm.*)

Operációs rendszer – memória menedzselés

Memória menedzselő komponensek funkciói:

- a)** Memória használatának nyilvántartása.
- b)** Memória *allokálás/felszabadítás* a processzek számára.
- c)** *Címleképzés: logikai* (virtuális) címek leképezése *valós* címekre:
 - ki/be söprés (swapping),
 - ki/be lapozás (paging).

Operációs rendszer – I/O menedzselése

I/O menedzsment funkciói:

a) *Eszköz driverek* biztosítása:

- blokkorientált eszközökhöz (diszkekhez),
- karakterorientált eszközökhöz (terminálok, nyomtatók, portok, stb.),
- speciális eszközökhöz (óra, hálózat, stb.)

b) *Szabad blokk menedzselés* (diszkeken)

Operációs rendszer – Fájlrendszer menedzselése

Fájlrendszert megvalósító komponensek funkciói:

a) *Directory struktúra* biztosítása

- Jegyzék implementáció, fájl-attribútumok rögzítése.
- *Blokk hozzárendelés* módszerei.

b) Fájlok *kreálása/törlése*.

c) Jegyzékek *kreálása/törlése*.

Operációs rendszer – Fájrendszer menedzselése

d) Fájlok *írása/olvasása*.

e) Fájrendszer létrehozásának, helyreállításának segédprogramjai.

f) Mentések és visszaállítások segédprogramjai.

Operációs rendszer – Fájlrendszer menedzselése

- *Védelmi komponens(ek) funkciói:*
 - Erőforrásokhoz való hozzáférésekkel kapcsolatos *nyilvántartások kezelése.*
 - *Jogosultságok betartatása.*

Operációs rendszer – felületek programozónak

B.) Milyen *felületeket* (interface) ad a *programozóknak*, *felhasználóknak*?

1. A *programozók felülete*:

- *rendszerhívásokat* (system calls): alkalmazás-programozási felület (Application Programming Interface, API)
- *Hardverből*: megszakítás, hibakezelés.

Operációs rendszer – felületek, API

Programozói felületeket:

- API (*Application Programming Interface*)
 - Léteznek szabványok: A forrás szinten való átvihetőséget biztosítják.
 - Klasszikus API-k: SVID, BSD (kaliforniai Berkeley egyetem)
 - *POSIX* szabvány

Operációs rendszer – felületek, API

1. **SVID** (System V Interface Definition)

Egységes felhasználói felületet és definícióit határoz meg.

Leírja az *AT&T UNIX System V* viselkedését,

- beleértve a rendszerhívásokat,
- C-könyvtárakat,
- elérhető programokat és eszközöket.

Operációs rendszer– felületek, API

2. POSIX (Portable Operating System Interface for Unix) szabvány, a Unix operációs rendszerek API-jának meghatározásaként definiált.

(nem csak UNIX, pl. Win NT (később), OpenVMS, stb.)

API - Application Programming Interface - kitérő

Mi az API fogalma?

Az API - egy interfész, kell mögé egy komponens,

- ami megvalósítja az *API-ban definiált függvényeket*,
- és a komponens, ami szolgáltatásokat biztosítja az *API-ban*.

API -Application Programming Interface - kitérő

„Kérdés: kívülről milyen függvényeken, adatstruktúrákon keresztül érhető el egy alkalmazás?

OS esetén: a rendszerhívások segítségével.

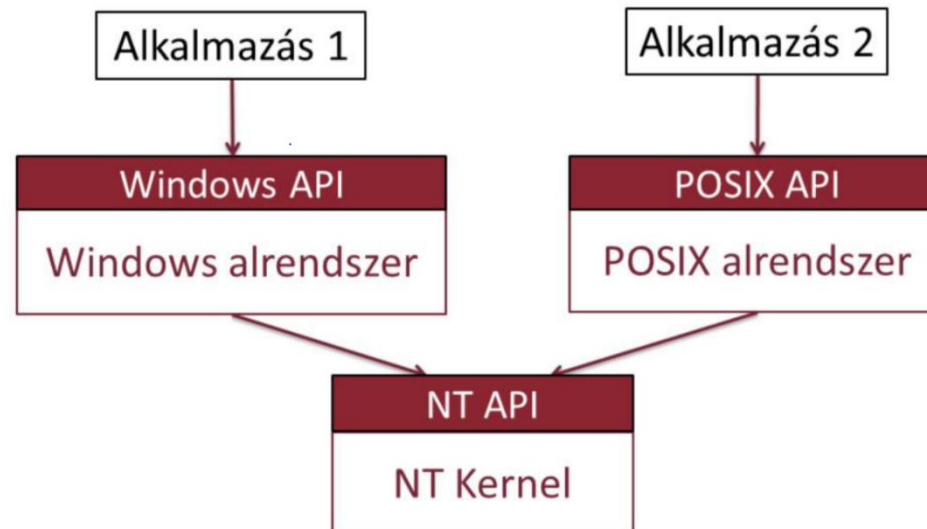
- Win32: CreateFile(), ReadFile(),...
- POSIX : fopen(), fork(), select()...

API (Application Programming Interface) - kitérő

Hogyan lehet a Windowsnak *Win32*, *POSIX* és *OS/2* API-ja?

Megoldás: környezeti alrendszer

API (Application Programming Interface) - kitérő



API (Application Programming Interface) – kitérő

A három különböző alrendszer nem csak *különböző nevű függvényeket* jelent (fopen vs. CreateFile), hanem teljesen *eltérő szemantikájuk* is.

Például:

- POSIX esetén a fájlnevben számít a kis és nagybetű.
- Windows esetén nem.
- POSIX szál vs. Windows-os szál – más tulajdonságok vannak.

API Application Programming Interface – kitérő

- Windows API és POSIX: teljesen dokumentált.
- NT API: nem dokumentált, csak egyes részei, ami szükséges az eszközfejlesztéshez.
- Windows API: régi neve Win32 API volt - 64 bit-es miatt lett.

Az alkalmazások viszont *nem keverhetik* az alrendszereket, mindegyik csak egyet használhat - ezt linkeléskor kell eldönteni.”

URL: <https://micskeiz.wordpress.com/2010/06/14/egyszeru-posix-alkalmazas-windows-alatt/>

Operációs rendszer – felhasználók felülete

2. *A felhasználók felülete:*

- *Parancsnyelvi,*
- *Grafikus kezelő felület* (burok, GUI)
- *Segédprogramok* készlete

(ezen processzek programjaiban API hívások vannak.)

API hívásokat használnak, tehát ráépülnek a kernel-re.

Operációs rendszer– felületek, UI

1. *Karakteres felhasználói felületek (UI)*

- *Parancssoros* (pl. UNIX burok, WIN cmd.exe, PowerShell, stb.)
- *Menü vezérelt* (pl. IBM AS/400) (kvázi történelem),

<https://www.youtube.com/watch?v=7980eoVjz08>

OS-ek felépítése – felületek, UI

2. *Grafikus felhasználói felületek (GUI)*

- *1973: Xerox Alto az első GUI és „egér”*
<https://www.youtube.com/watch?v=M0zgj2p7Ww4>
- *80-as évek: Apple Macintosh, Microsoft Windows, X11 Window System.*
- *Napjainkban: X11 (X.Org, stb.), MacOS, Windows NT, Windows 11, Linux Mint stb.*

OS-ek felépítése – felületek, UI

Parancssoros felület - karakter alapú:

- Készenléti jelet ad.
- Parancssort dolgoz fel.
- Legsikeresebb parancs feldolgozók: UNIX shell-ek. (hég egy felhasználói felület, amely által hozzáférhetünk az operációs rendszer szolgáltatásaihoz. A rendszerhéjak parancssori felületet (CLI))

OS-ek felépítése – felületek, UI

Menü vezérelt felület

- *Karakteres alapon menüpontokból választhatunk.*
- *Példa: Az IBM AS/400 -as kombinált parancssoros és menü-vezérelt felületé.*

OS-ek felépítése – felületek, UI - AS/400 menü

```
MAIN                                AS/400 Main Menu                                System:  JVCDEV
Select one of the following:
    1. User tasks
    2. Office tasks
    3. General system tasks
    4. Files, libraries, and folders
    5. Programming
    6. Communications
    7. Define or change the system
    8. Problem handling
    9. Display a menu
   10. Information Assistant options
   11. Client Access/400 tasks

   90. Sign off

Selection or command
==> crtoutg

F3=Exit  F4=Prompt  F9=Retrieve  F12=Cancel  F13=Information Assistant
F23=Set initial menu

MR  b                                MW                                20/014
```

<https://rpgcorner.blogspot.com/2005/12/how-to-display-first-menu-to-user.html>

OS-ek felépítése – felületek, UI

Grafikus felhasználói felületek (GUI)

Többféle van, de alapjaik közösek:

- *Ablak* (Window): „bármely” téglalap alakú része a képernyőnek.
- *Asztal* (Desktop): a képernyőn megvalósuló *munkaterület* (ez is ablak).
- *Mutató* (Pointer): spec. beviteli eszközzel (egérrel) mozgatható kiválasztó.

OS-ek felépítése – felületek, UI

- *Ikon* (Icon): négyzet alakú ablak, amely egy objektumot szimbolizál.
- *Objektum* (Object): mappa, alkalmazás, állomány.
- *Parancs* (Command): kattintások, billentyű leütések.

OS-ek felépítése – segédprogramok

Ismert segédprogram *Sysinternals Suite* csomag.

A Windows belső működését lehet tanulmányozni, vagy a hibakeresésben segít.

<https://docs.microsoft.com/hu-hu/sysinternals/downloads/sysinternals-suite>

OS-ek felépítése – segédprogramok

A Sysinternals weboldalon kategóriákba sorolva hasznos programok érhetők el:

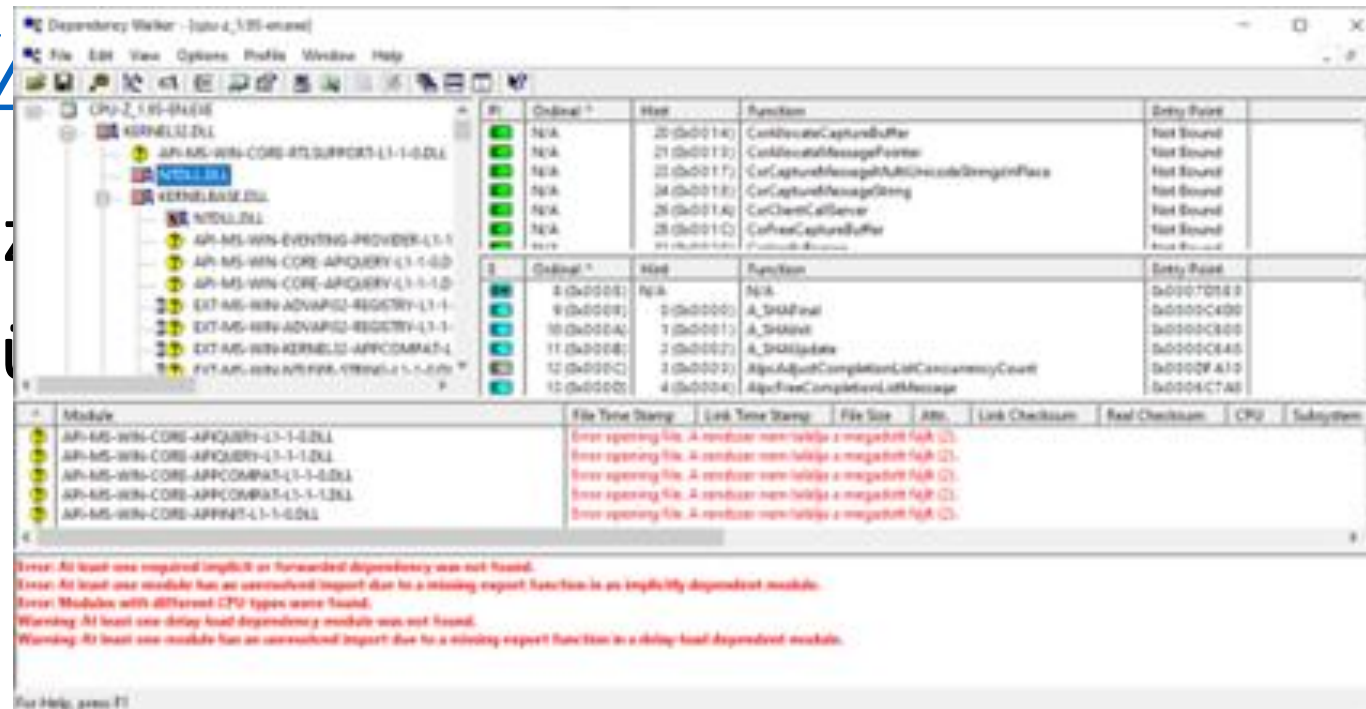
- a)** File and Disk Utilities (Disk2vhd)
- b)** Networking Utilities (TCPView)
- c)** Process Utilities (Process Explorer, Process Monitor, AutoRuns)
- d)** Security Utilities (LogonSession)
- e)** Information Utilities (RAMMap)

OS-ek felépítése – segédprogramok

Dependency Walker

URL: <http://www.dependencywalker.com/>

Feladata: ez
és azon belüli
program.



nappakra,
indított

OS-ek felépítése – segédprogramok Linux OS

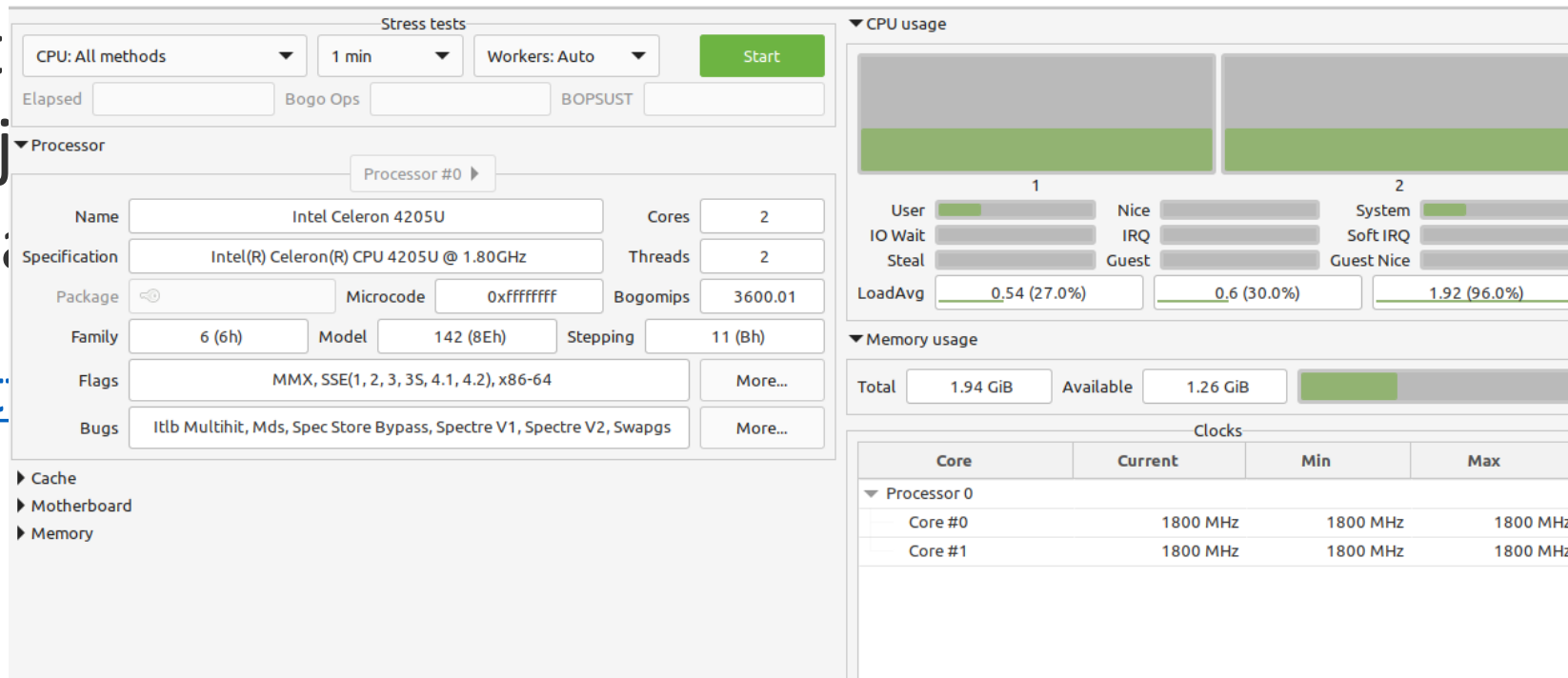
Grafikus rendszer monitorozó – **GTKStressTest**

A un. St

használj

Szolgált

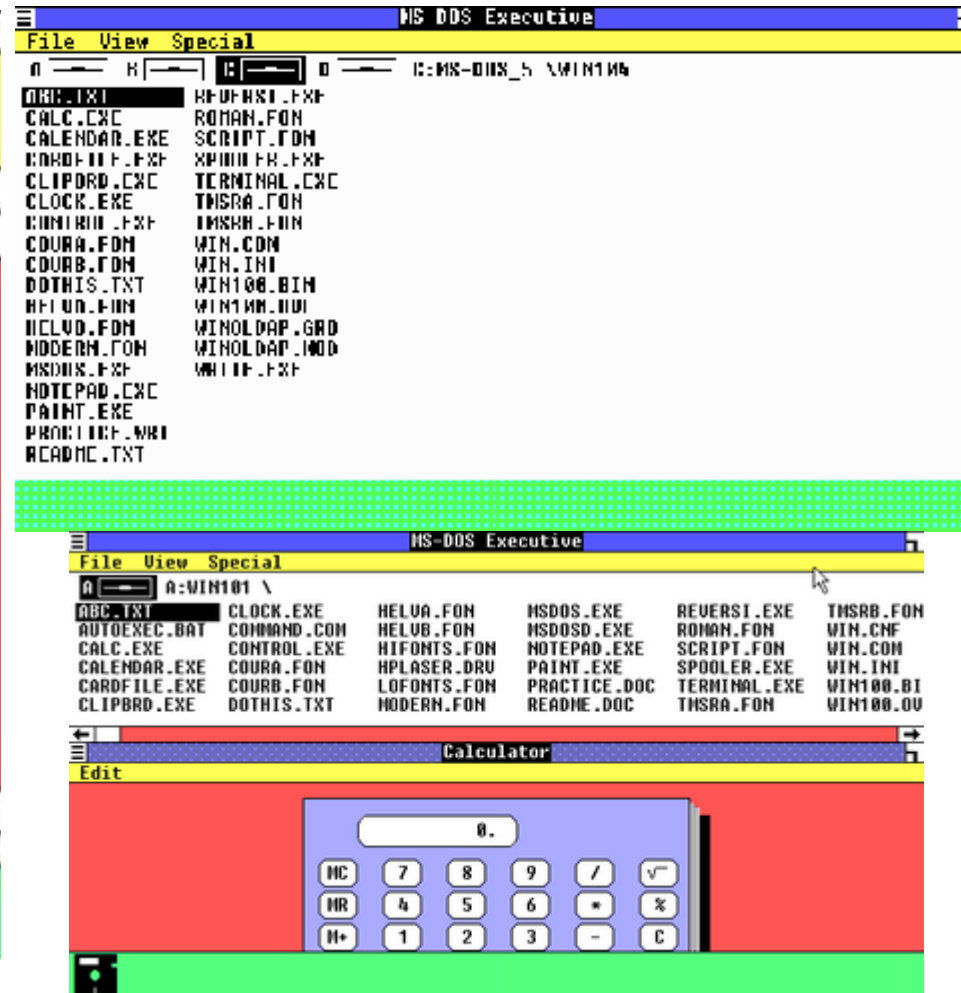
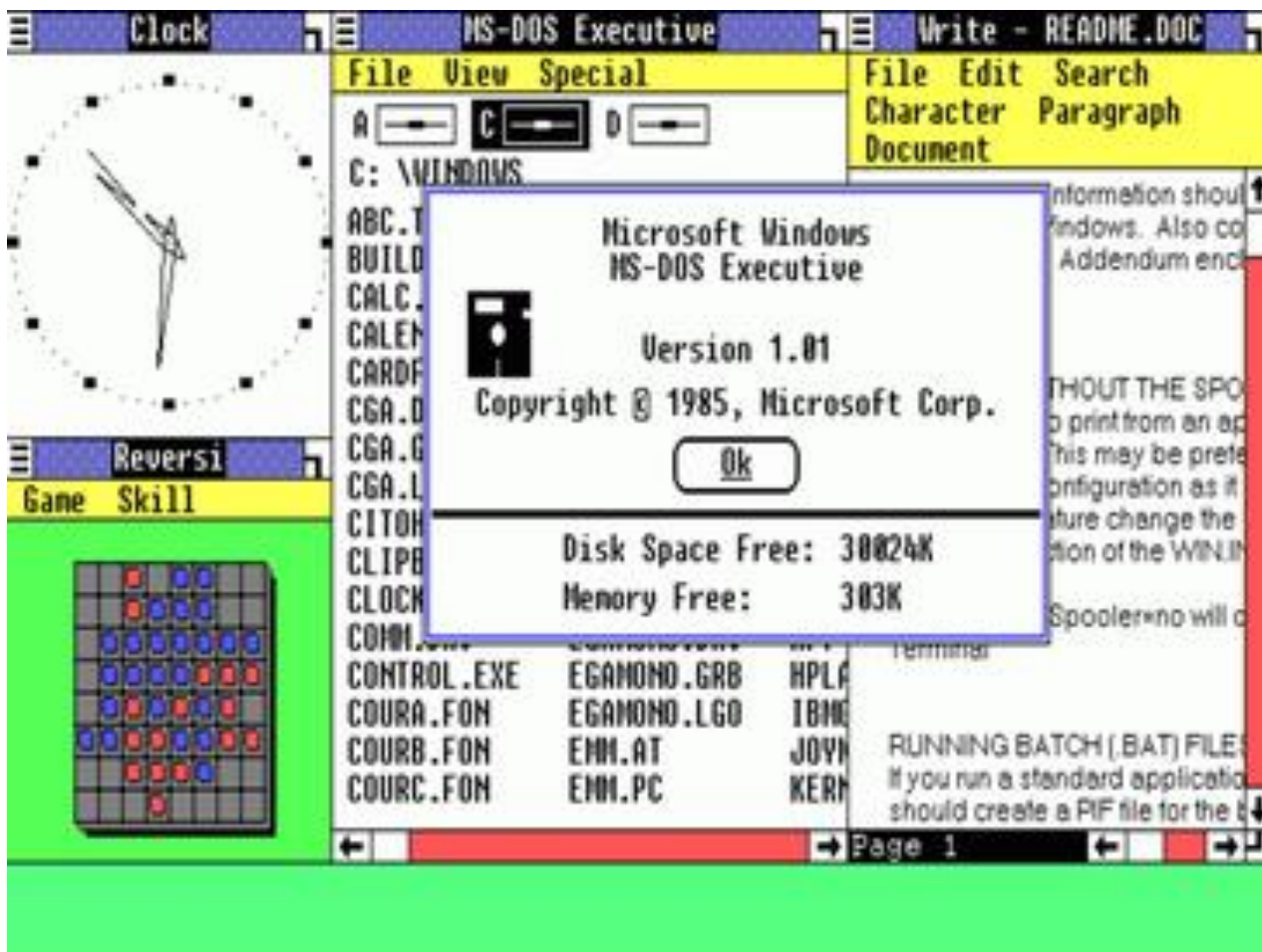
- URL: <http://www.gtk-stress-test.com/>



tén is

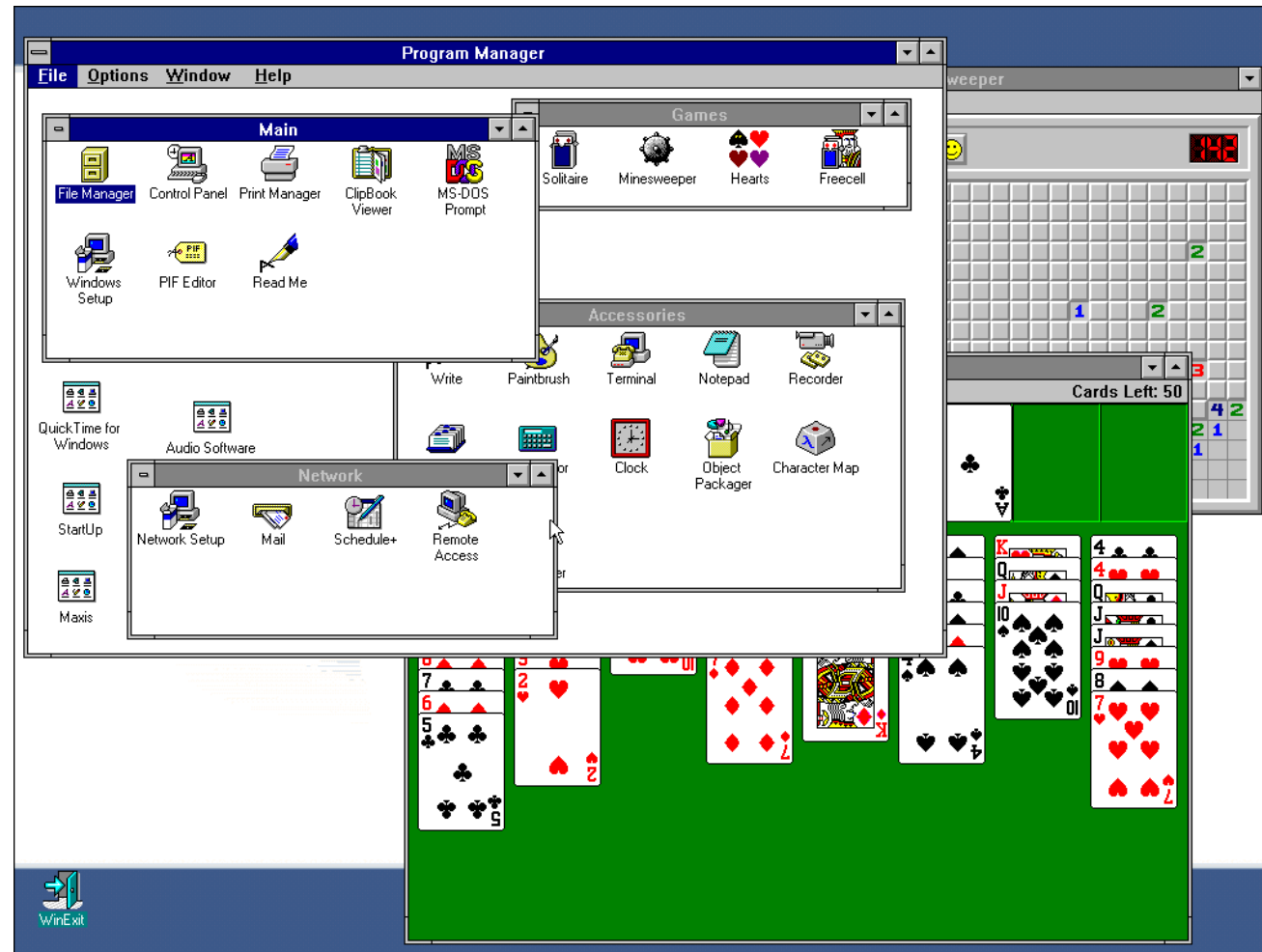
<http://www.gtk-stress-test.com/>

OS-ek felépítése – felületek, GUI példák (Win 1.01)

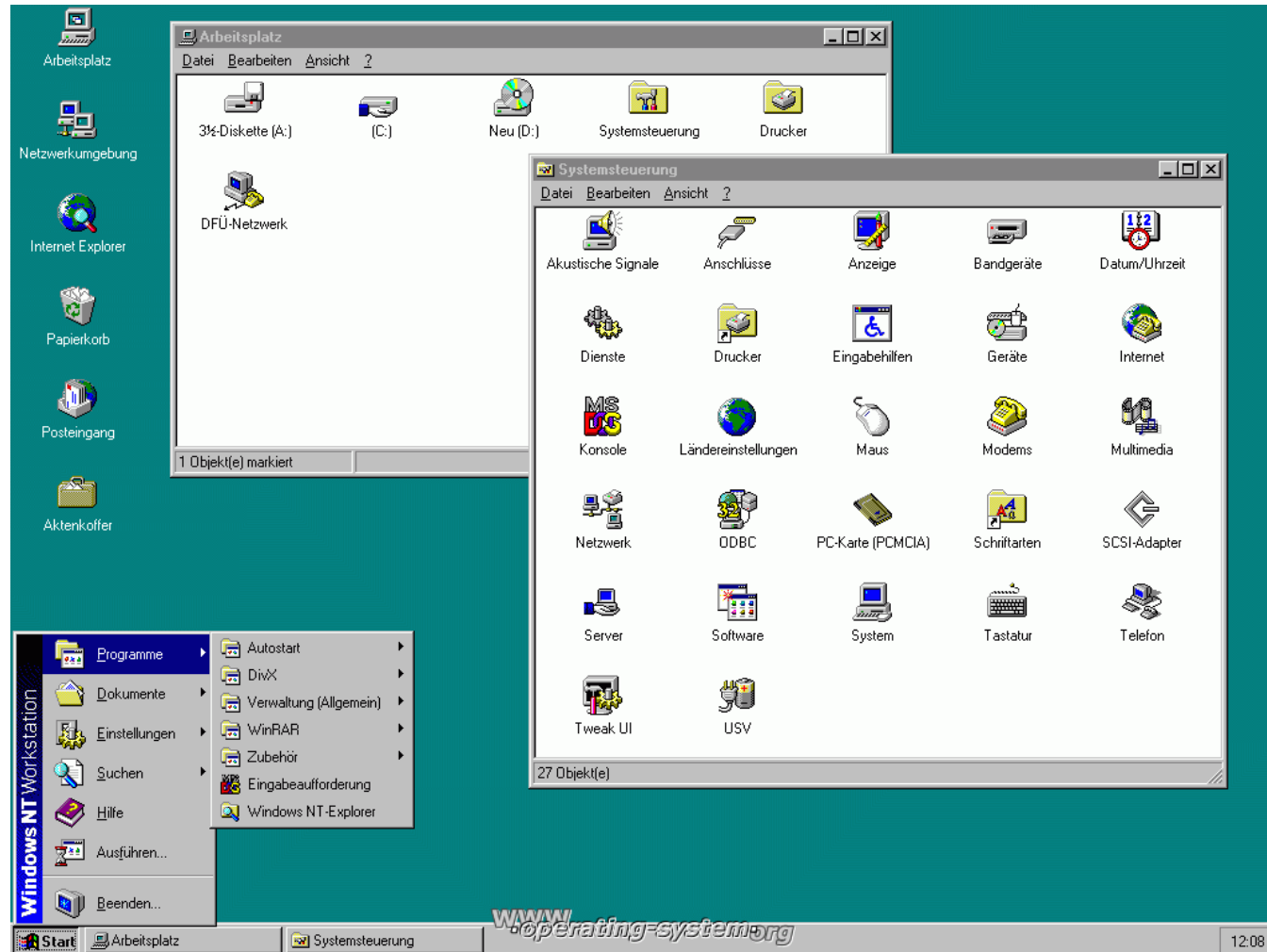


https://en.wikipedia.org/wiki/Windows_1.0

OS-ek felépítése – felületek, GUI példák (Win 3.11)



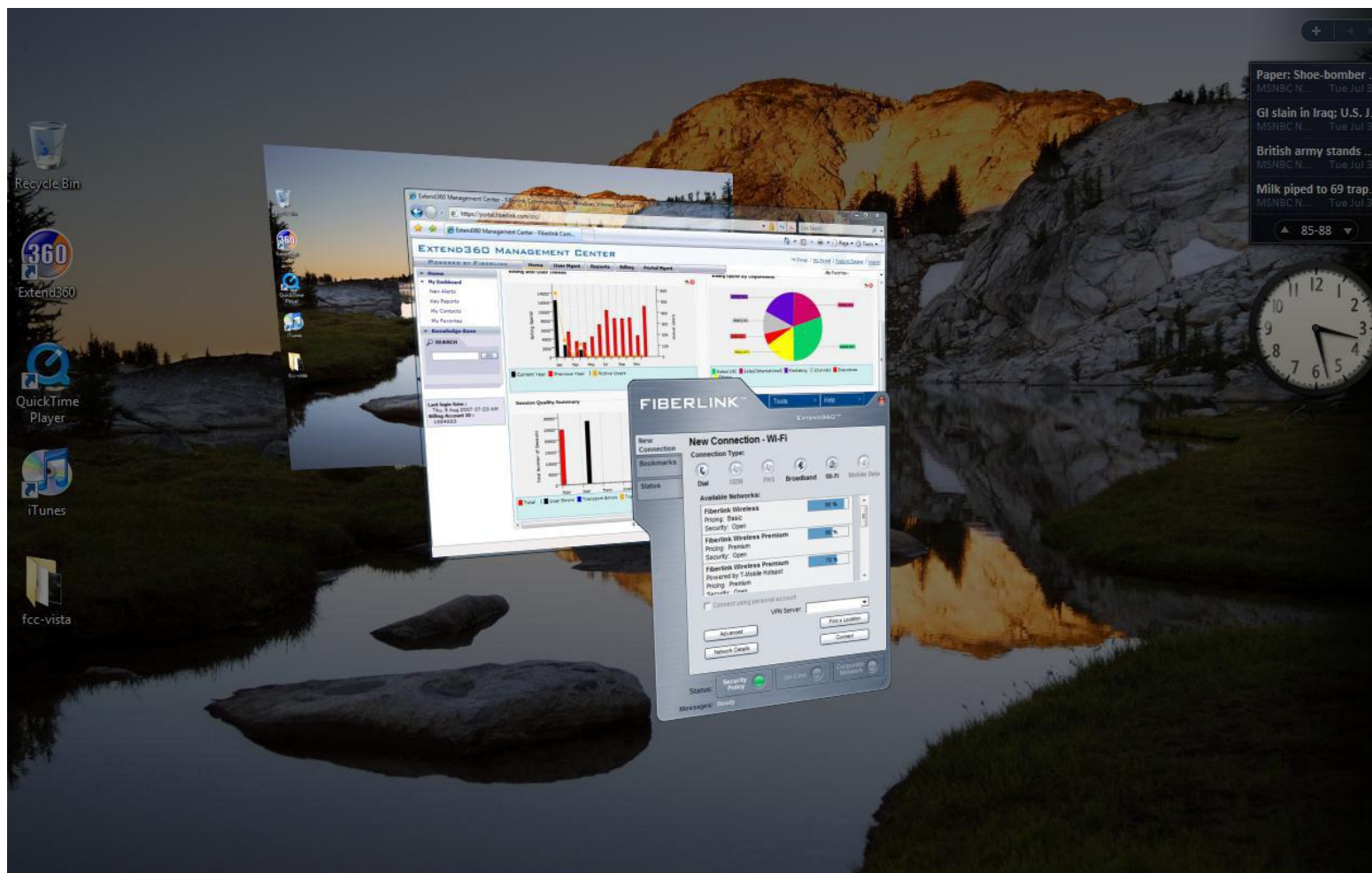
OS-ek felépítése – felületek, GUI példák (WinNT 3.51)



OS-ek felépítése – felületek, GUI példák (WinXP)

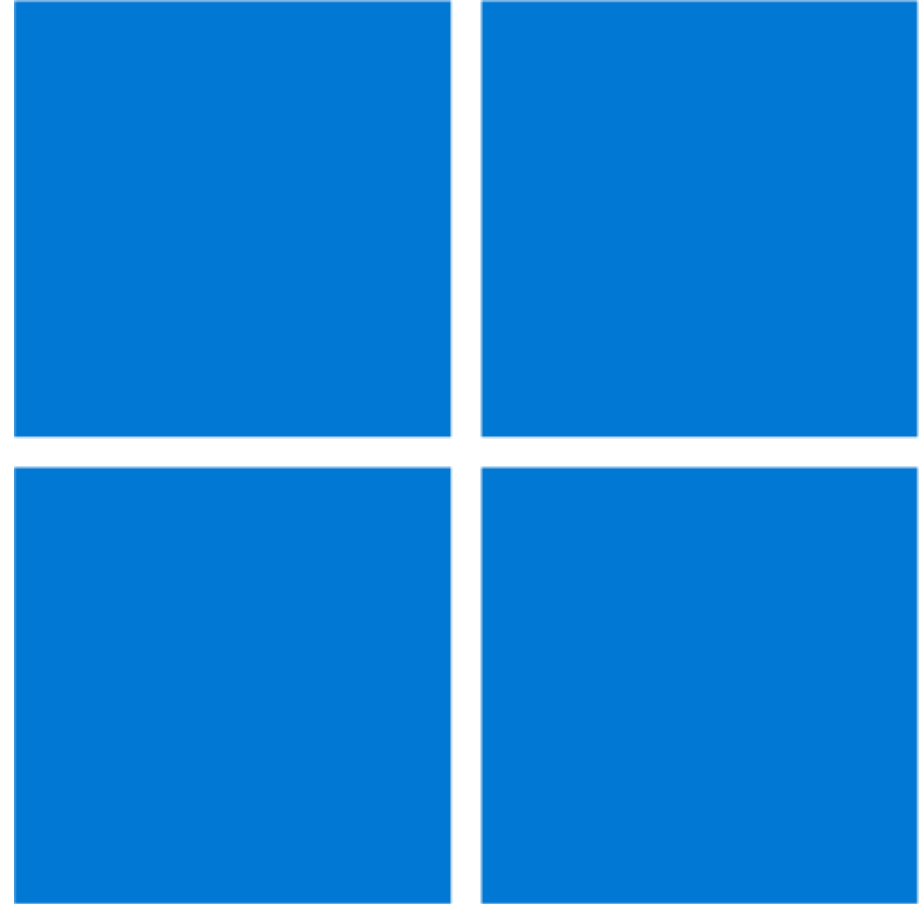
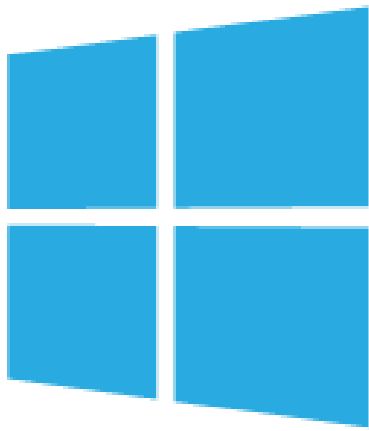


OS-ek felépítése – felületek, GUI példák (Vista)



<https://www.hwsz.hu/hirek/57053/microsoft-windows-vista-operacios-rendszer-tamogatas-vege.html>

OS-ek felépítése – felületek, GUI példák (Windows 11)

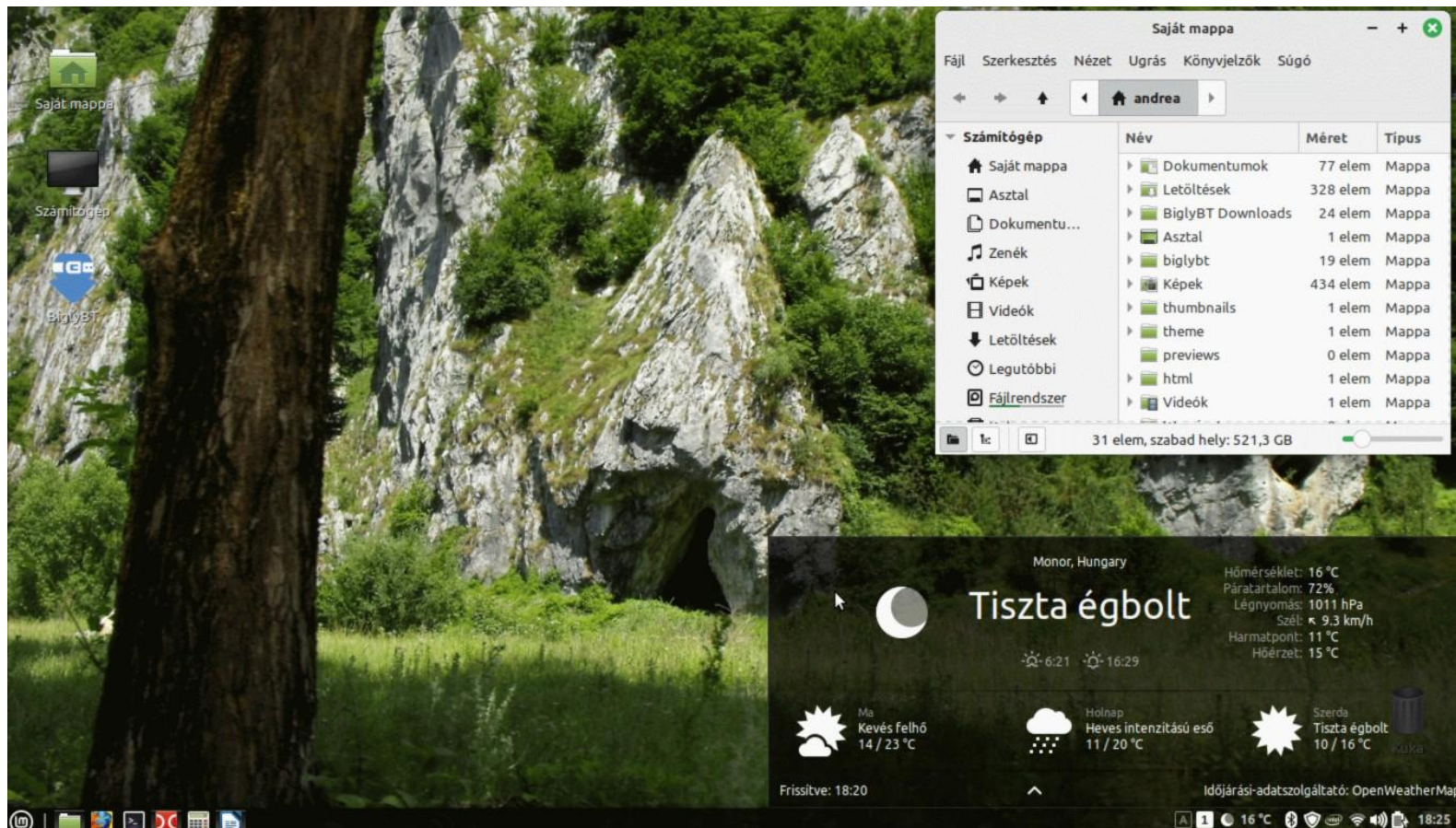


Windows 11

<https://www.techspot.com/downloads/7437-windows-11-installation-tool.html>

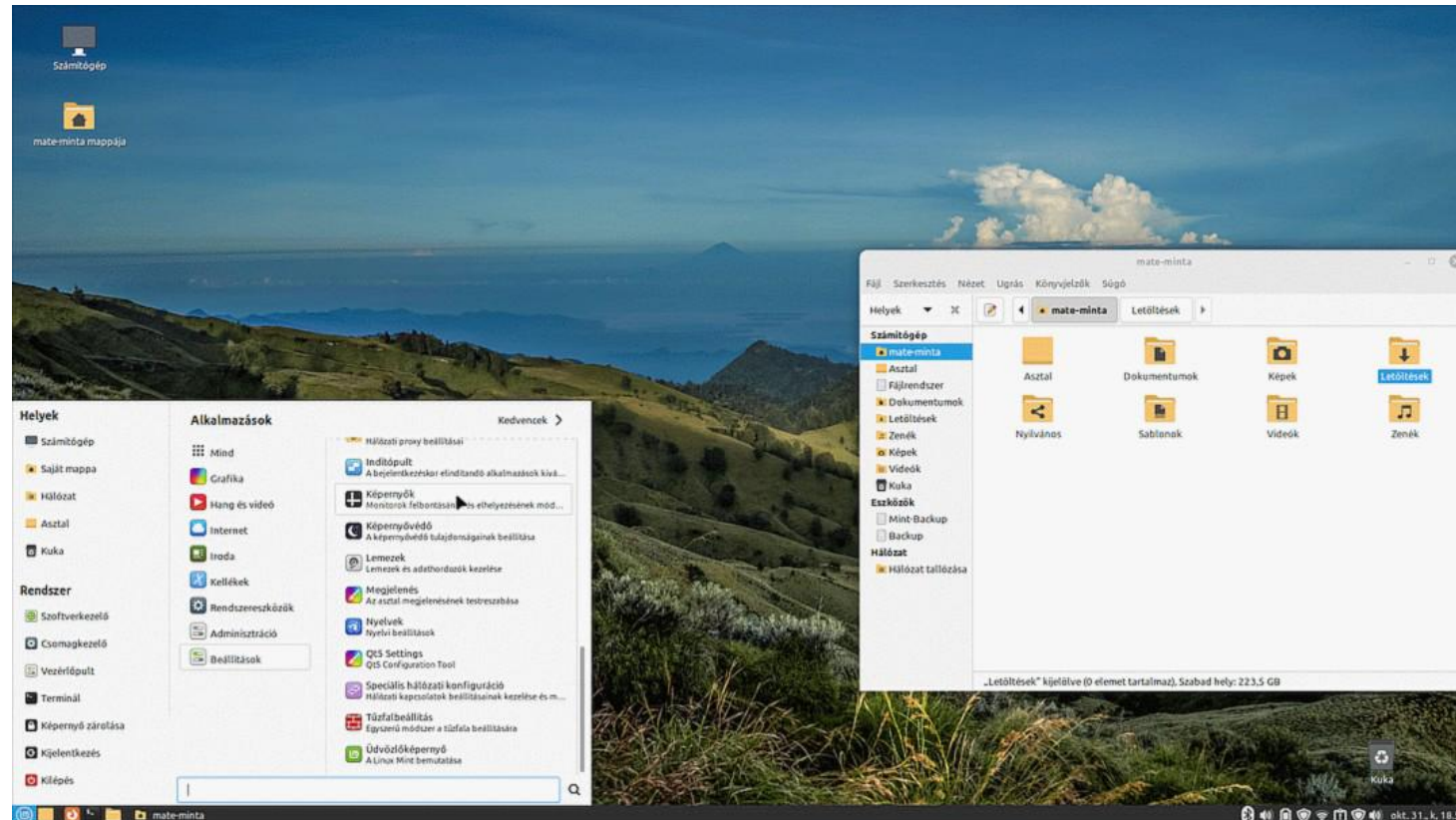
OS-ek felépítése – felületek, Linux Mint

A **Cinnamon** *felhasználói* felület a Mint csapat zászlóshajója.



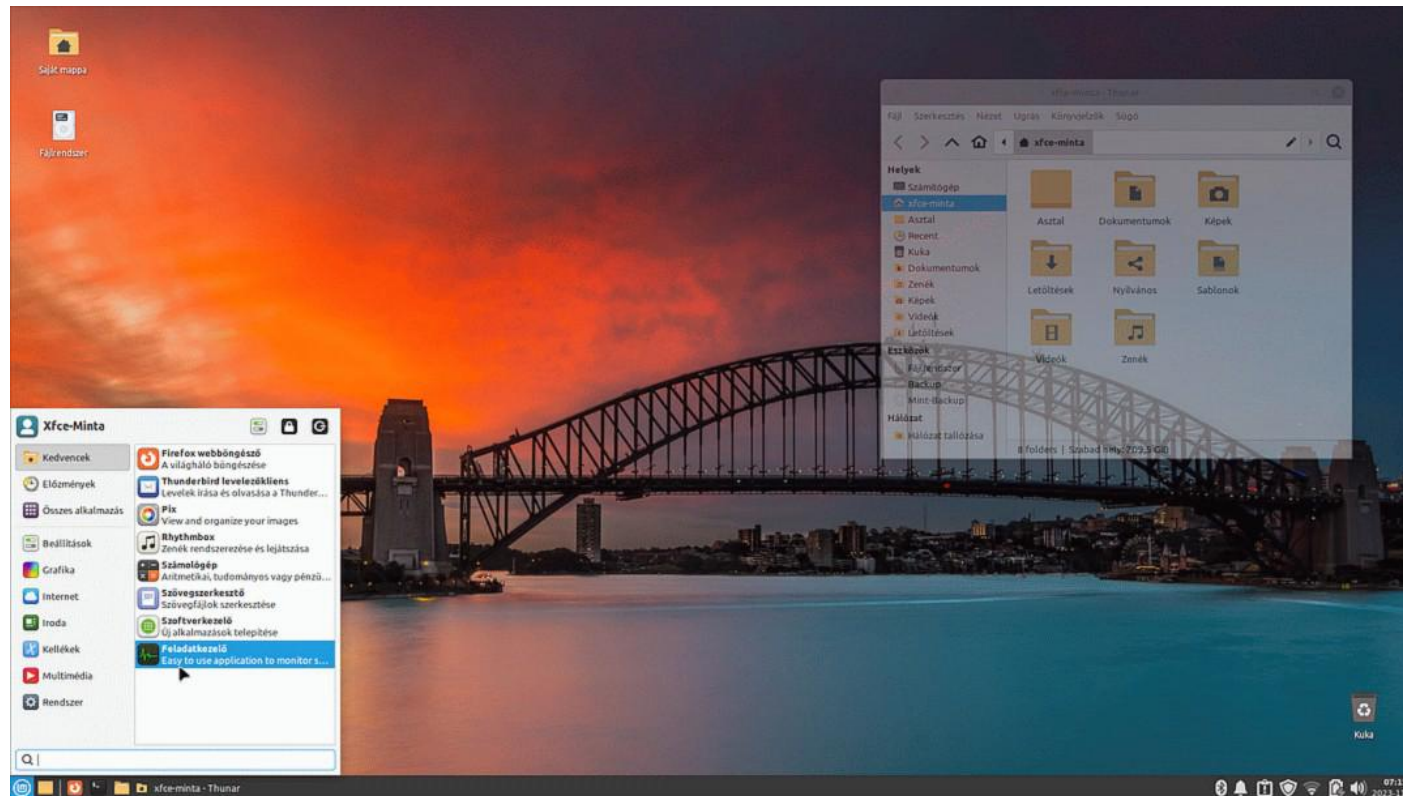
OS-ek felépítése – felületek, Linux Mint

A **MATE** legfontosabb tulajdonsága a 3 oszlop széles menüje.



OS-ek felépítése – felületek, Linux Mint

Xfce (XForms Common Environment) - elvileg ez a legkisebb *erőforrást igénylő* felület, ezért a gyengébb gépekre ajánlják.



OS-ek felépítése – felületek, (Mac OS 9)

1999. Apple
klasszikus MacOS
operációs rendszere.



<https://www.macintoshrepository.org/6660-mac-os-9-spanish->

OS-ek felépítése – felületek (OS X Leopard)

Mac OS X Leopard a
*Az Apple asztali és
szerver OS hatodik
legfontosabb kiadása*
Macintosh
számítógépekhez.



<https://isriver.com/mac-os-x-leopard-10-5-iso/>

OS-ek felépítése – felületek, (iOS)



<https://pcworld.hu/macworld/az-ios-rendszerek-nagy-ujitasai-155331.html>

OS-ek felépítése – Implementációs struktúra

3. *Implementációs struktúra* – kernel. Komponenseit hogyan állítják elő, köztük milyen interfészek vannak?

- Monolitikus rendszer.
- Réteges struktúrák.
- Virtuális gépek.
- Kliens-szerver modell (mikrokernel).
- Hibrid szerkezet.

Monolitikus kernel – jellemzői

A rendszermag egyetlen modulból áll.

- Nincs strukturáltság.
- *Kernel (mag): egy modulba összelinkelt, szolgáltatató rutinok, utilities, megszakítást kiszolgáló rutinok összessége.*
- *A kernel a rendszer indításkor (teljes egészében) betöltődik.*

Monolitikus kernel – jellemzői

- A monolitikus rendszer *rutinjait assembly*, esetleg *magas szintű nyelven* írják.
- Lefordítják (compile) és összeszerkesztik (linkelik) egyetlen *betölthető programba*, lementik egy fájlba.
- Ez a fájl a rendszerindításkor betöltődik és a kernel ott áll szolgáltatásra készen.

Monolitikus kernel– jellemzői

Hogyan valósul meg a *szolgáltató rutinok* hívása:

- *rendszerhívással* (system call) – explicit (nyílt, közvetlen),
- *megszakítással* (IT) – implicit (rejtett, burkolt), vagy
- *kivételes eseményre való reagálással*.

Futási mód váltás (user mód - kernel mód)

Tipikus példa: régebbi Linuxok (2.0-nál régebbi) és UNIX rendszerek jellemző.

Monolitikus kernel - hívások

A monolitikus rendszer magja *szolgáltató rutinok gyűjteménye*, melyek *hívhatók*:

- *felhasználói programból* (processzből) - *rendszerhívással* (*kernel call, system call*),
- és egy *szolgáltató rutinból* (call).

Monolitikus kernel - hívások

A két hívási mód megkülönböztetésének okai:

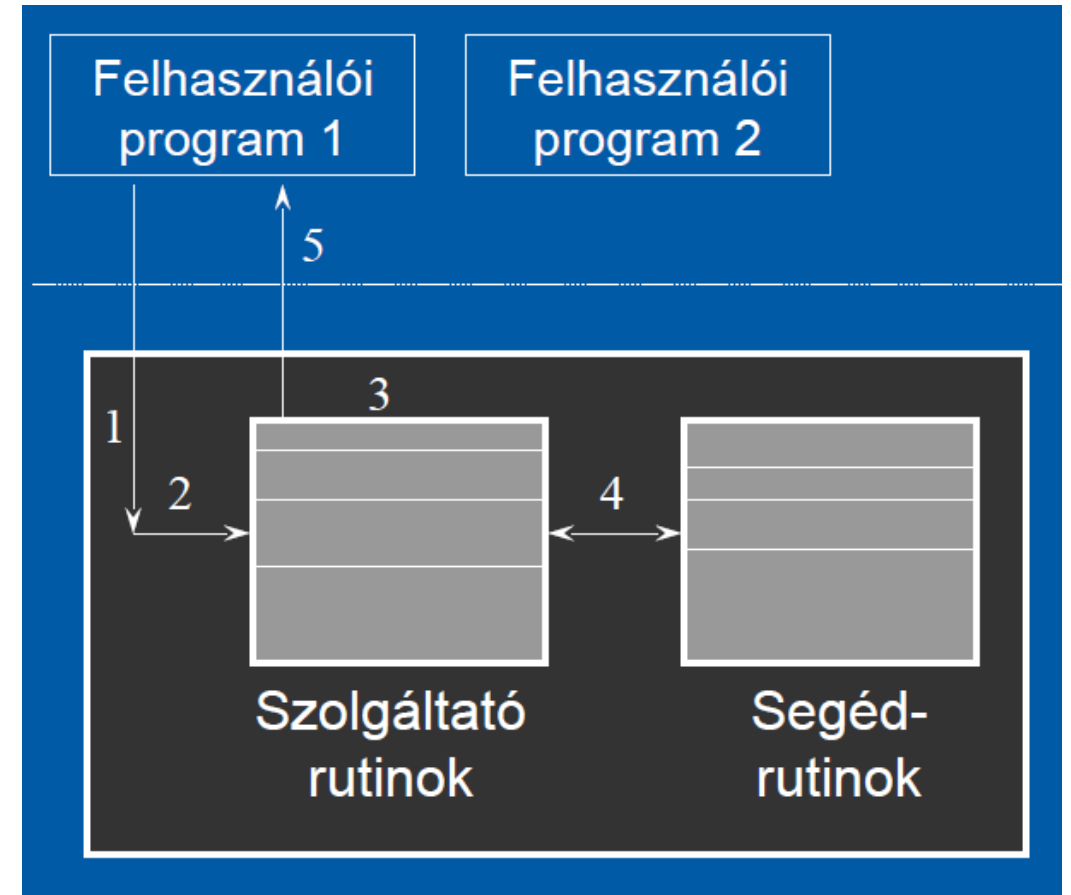
a. A *system call* egy *függvényhívás*: meg kell adni a *szolgáltató rutin nevét* és *aktuális paramétereit*.

- *De nemcsak, hanem egyben trap is*: a processzor *user módból - kernel módra* vált.

b. *Call hívás* esetén a *kernel szolgáltató rutinjai* egymást korlátlanul hívhatják – *nem szükséges a módváltás*.

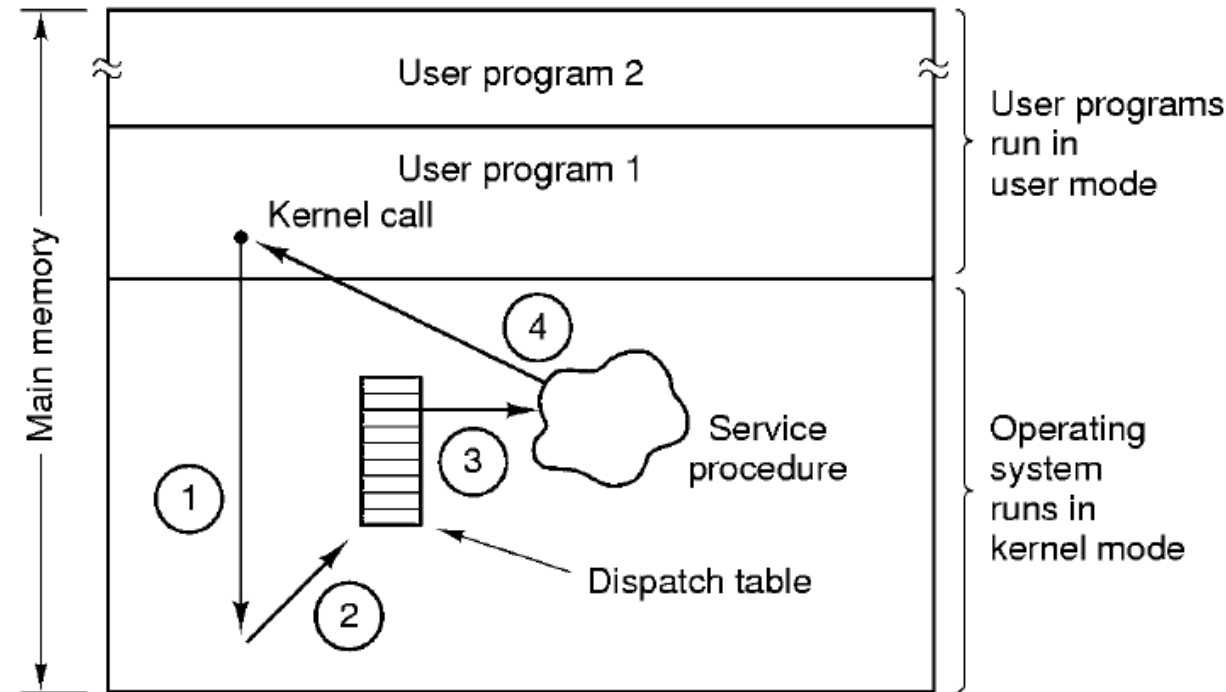
Monolitikus kernel felépítése

1. Rendszerhívás.
2. Paraméter ellenőrzés.
3. Szolgáltató rutin végrehajtás.
4. Rutin segédrutint hív/visszatér.
5. Visszatérés rutinból.
(vissza az eredeti priv. szintre).



Monolitikus kernel - system calls

1. A hívó proc-nak joga volt-e.
2. Hívás paraméterek ellenőrzése.
3. A táblázat a rendszer *eljárások kezdő címet* tárolja.
4. Vezérlés visszaadása a proc-nak.

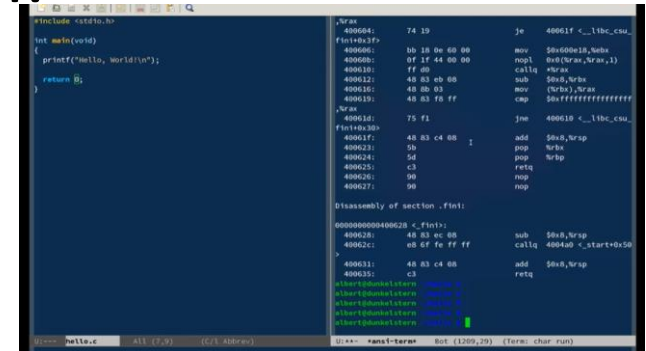


Monolitikus rendszer

Monolitikus rendszer

- Konkrét rendszerhívás implementáció Linuxon:
 - **64-bit:** syscall instrukció

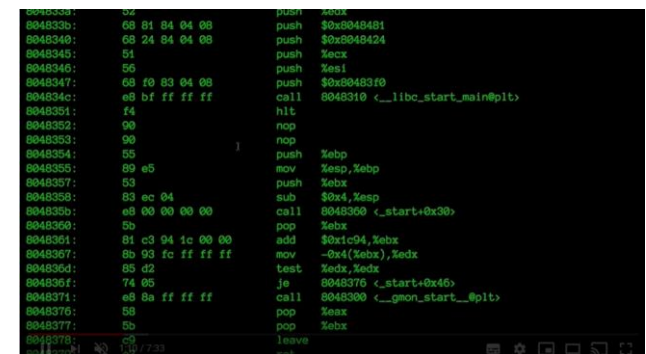
<https://www.youtube.com/watch?v=mB79rNrpOhg>



The screenshot shows a debugger window with two panes. The left pane displays C code for a function named 'main' that prints 'hello, world!'. The right pane shows the corresponding assembly code. The instruction at address 400620 is 'syscall', which is the 64-bit system call instruction used to invoke the kernel's system call interface.

- **32-bit:** int 0x80 – soft interrupt
instrukció 80h interrupt handler

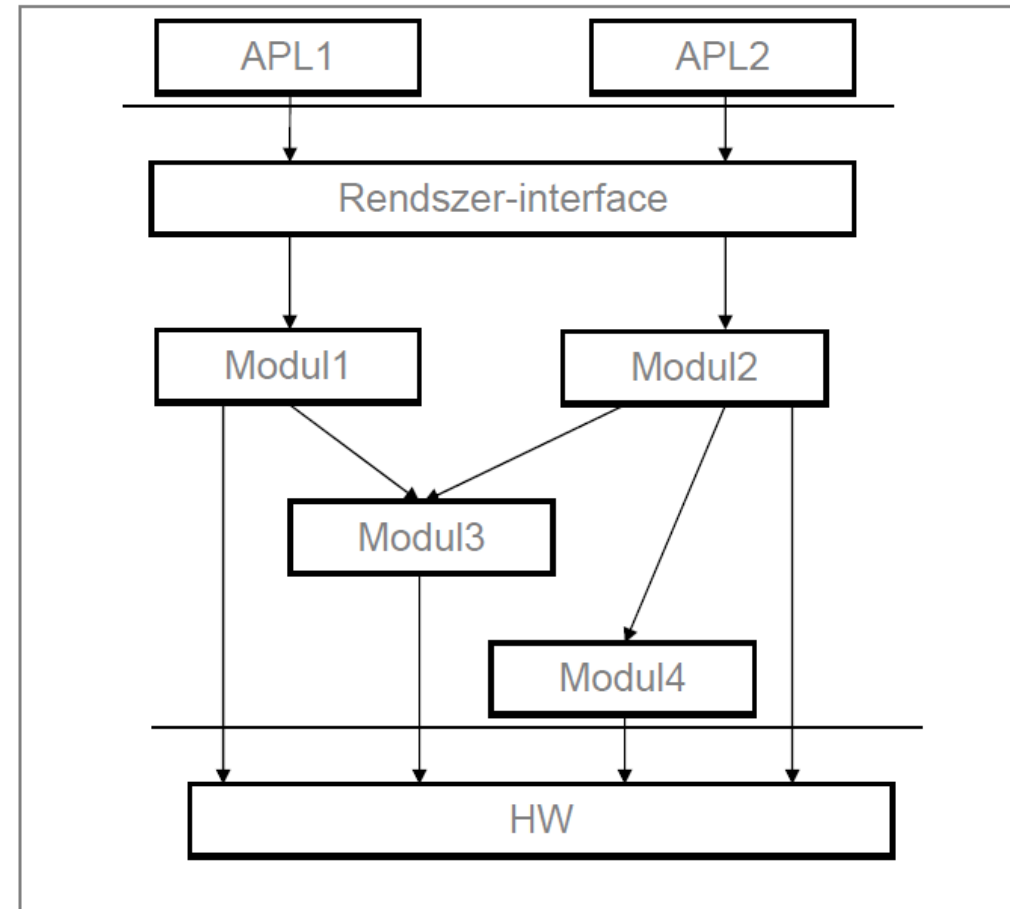
<https://www.youtube.com/watch?v=HdQlwdj2BXc>



The screenshot shows a debugger window with two panes. The left pane displays assembly code for a function named 'main'. The right pane shows the corresponding assembly code. The instruction at address 8048370 is 'int \$0x80', which is the 32-bit system call instruction used to invoke the kernel's system call interface.

Monolitikus kernel - felépítése

- Közvetlen *kommunikáció modulok* között.
- "*Közösen*" használt adatok a *közös memórián*.
- Nincs szabványos *érintkezési felülete* a moduloknak (interface)

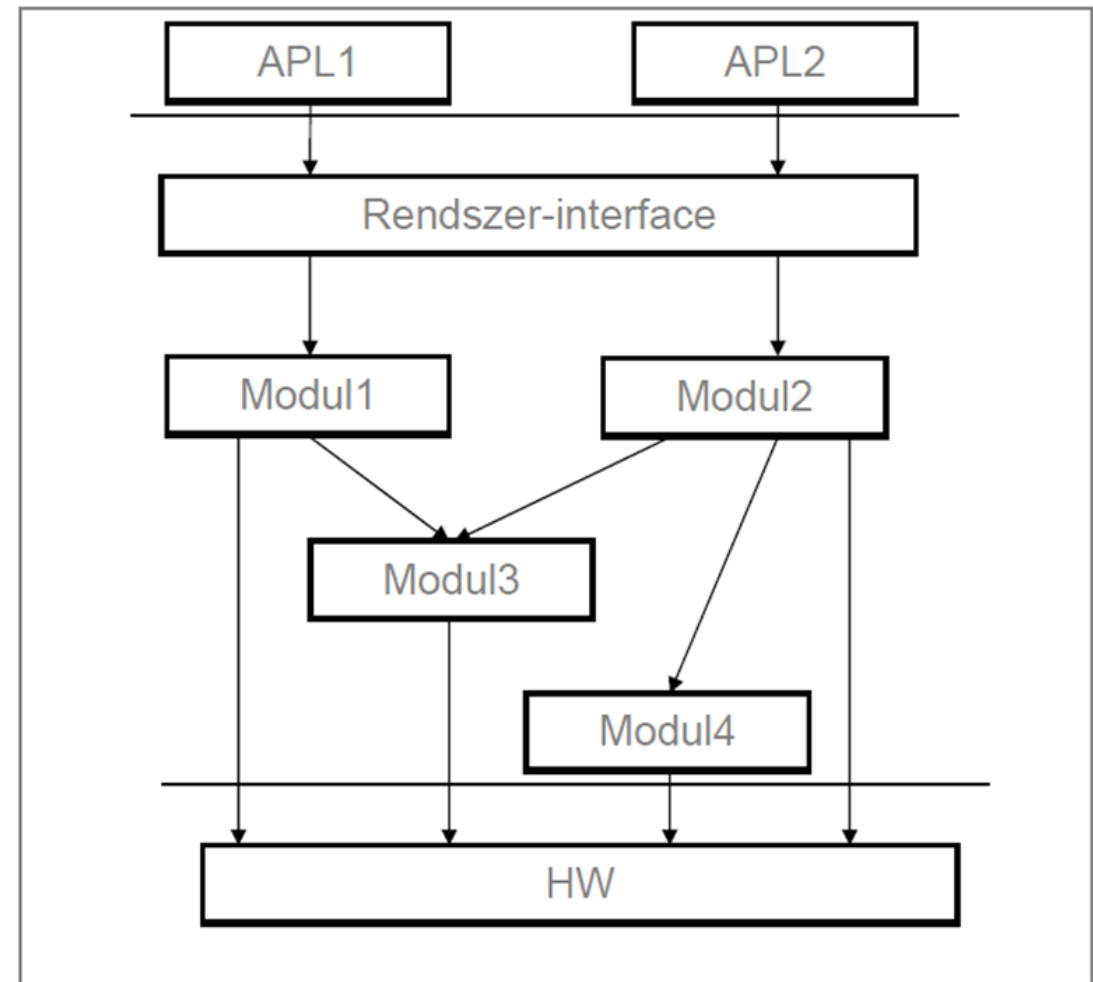


Monolitikus kernel - felépítése

- *Nem megbízható*: egy hibás modul hibája tovább terjedhet a többi modul felé.
- Nehezen karbantartható, fejleszthető.

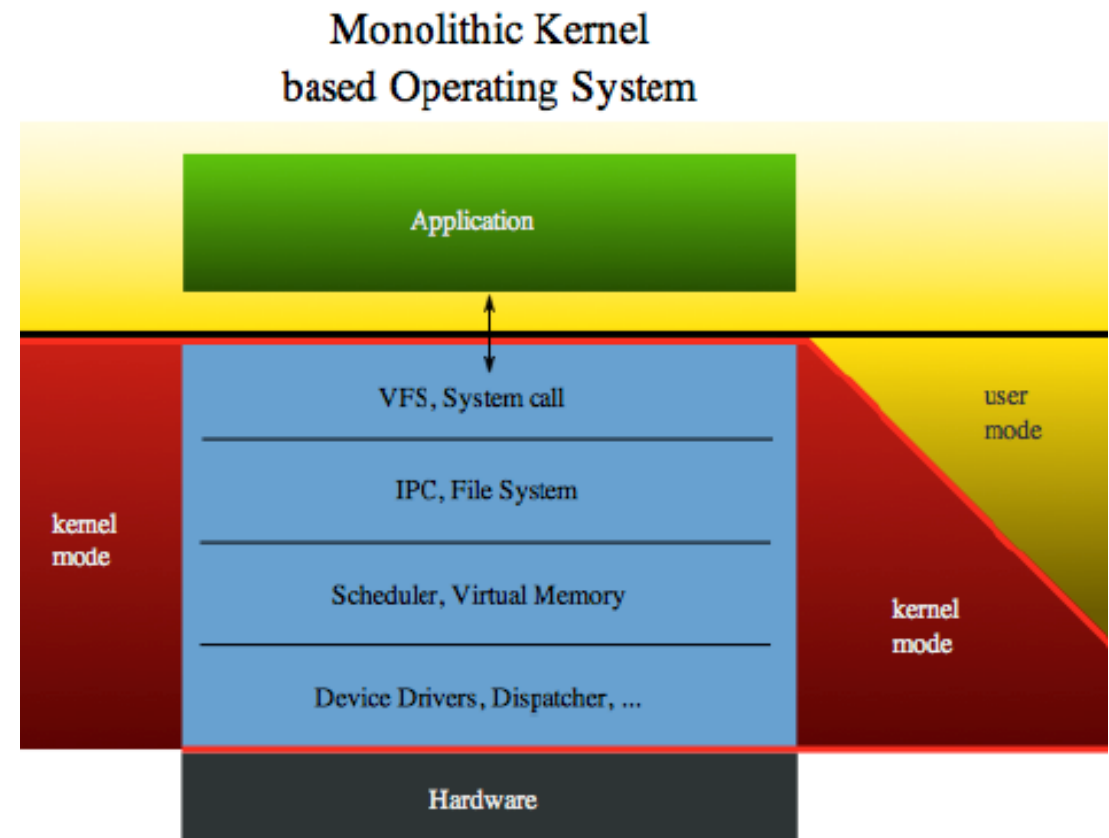
További információk: URL:

<http://www.egalizer.hu/informatika/essze/mkernel.html>



Monolitikus kernel - felépítése

- A monolitikus kernelű operációs rendszer (mint pl.: a UNIX, Linux) *magja egyetlen programból áll.*
- A programban az *eljárások szabadon* hívhatják egymást, a köztük levő kommunikáció *eljárás paramétereiken* keresztül zajlik.



Rendszerhívások - System calls

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

Réteges struktúrák

- 1968-ban *E.W. Dijkstra* (Edsger Wybe Dijkstra) *professzor* (holland matematikus, informatikus) és hallgatói készített egy **THE** nevű operációs rendszer (1968).
- Bevezette a *strukturált programozás* fogalmát az informatikában.
- A **THE** egyszerű *kötegelt feldolgozási rendszerű* operációs rendszer (az operátor csoportosítja a munkákat, ill. egyes fázisait = kötegelt feldolgozás (batch-ek)).

Réteges struktúrák

*A szolgáltató rutinok un. **rétegekbe** szervezettek.*

A rétegezés előnye:

- Egy rétegben *jól meghatározott, összetartozó funkciók vannak.*
- Magasabb szintű operációkat biztosít a felette lévő számára – *virtuális gépet.*
- Egy réteg *elrejt*i az alatta lévő *komponensek* részleteit.

Réteges struktúrák

A rétegezés hátrányai:

Hatékonyság probléma (alsó rétegben megvalósított szolgáltatás kivezetése)

A rétegeket külön-külön betölthető fájllokba linkelik:

- A betöltés akár dinamikusan is történhet (DLL).
- *Lehetséges:* a rétegekhez egyre privilegizáltabb futási módok.

Réteges struktúrák - THE struktúra

Dijkstra professzor és hallgatói készítették, 1968

(A rétegezés tervezési koncepció volt: egyetlen modulba linkelték.)

5	Operátor	Az operátor indít, lelő processzeket
4	Felhasználói programok	Független processzek
3	I/O menedzsment, eszközök	Felette virtuális (absztrakt) I/O
2	Operátor-processz kommunikáció	A processzeknek saját, virtuális konzolok
1	Memória-dob menedzsment	Virtuális memória, a processz látja a teljes címtartományát
0	Processz allokálás, multiprogramozás	Virtuális CPU-k, minden processztől elrejtve a többi

(További részletek a jegyzetbe...)

Réteges struktúrák - THE struktúra

0. réteg kiosztja a CPU-t köztük.

1. réteg: egy-egy részben a háttér

2. réteg: a processz

3. réteg: I/O kezelés számára.

Dijkstra professzor és hallgatói készítették, 1968 (A rétegezés tervezési koncepció volt: egyetlen modulba linkelték.)		
5	Operátor	Az operátor indít, lelő processzeket
4	Felhasználói programok	Független processzek
3	I/O menedzsment, eszközök	Felette virtuális (absztrakt) I/O
2	Operátor-processz kommunikáció	A processzeknek saját, virtuális konzolok
1	Memória-dob menedzsment	Virtuális memória, a processz látja a teljes címtartományát
0	Processz allokalás, multiprogramozás	Virtuális CPU-k, minden processztől elrejtve a többi

a CPU-t

alás,

káció.

processz

Réteges struktúrák - THE struktúra

4. rétegben találhatók a *felhasználói programok* – független processzek.

5. rétegben az egyetlen *operátor* alatt a *felhasználói programok* állnak. Őket.

A rétegek közötti interfész hívások.

Dijkstra professzor és hallgatói készítették, 1968 (A rétegezés tervezési koncepció volt: egyetlen modulba linkelték.)		
5	Operátor	Az operátor indít, lelé processzeket
4	Felhasználói programok	Független processzek
3	I/O menedzsment, eszközök	Felette virtuális (absztrakt) I/O
2	Operátor-processz kommunikáció	A processzeknek saját, virtuális konzolok
1	Memória-dob menedzsment	Virtuális memória, a processz látja a teljes címtartományát
0	Processz allokalás, multiprogramozás	Virtuális CPU-k, minden processztől elrejtve a többi

Réteg szerkezetű operációs rendszer

- A modulok *rétegszerűen épülnek egymásra* (objektum-orientált megközelítés).
- A kommunikáció *jól definiált interface-eken* keresztül bonyolódik.
- A rétegek csak a *felettük*, ill. *alattuk levő réteget* érik el közvetlenül.

Réteg szerkezetű operációs rendszer

- Az *interface*-ek használata:
 - *biztonságos*: ellenőrizhető a *kérés/hozzáférés jogossága*,
 - *megbízható*: nem terjed a hiba,
 - *karbantartható*: fejlesztés, módosítás egyszerű.

Virtuális gép - története

Az első és legismertebb az IBM VM/370 rendszere (1970).

A 60-as években kibocsájtott OS/360 rendszer *köteget feldolgozási* rendszer volt.

A felhasználók igényelték az *időosztást*, az IBM által kibocsájtott TSS/360 - túl nagy és lassú volt.

Egy fejlesztő csoport, IBM's Scientific Center, Cambridge MA, egy új megoldást mutatott be.

Ez volt a Virtual 370 koncepció: *a VM/370*.

Virtuális gép - története

Elgondoláson alapja (időosztásos rendszer) az, hogy:

- biztosítsa a *multiprogramozást*,
- biztosítson egy *kiterjesztett gépet*.

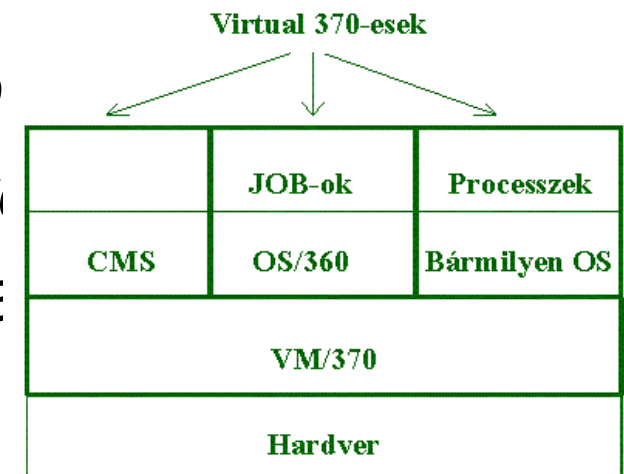
A két követelményt a VM/370 rendszer teljesen szétválasztva biztosítja.

(További részletek a jegyzetbe...)

Virtuális gép

A rendszer lelke a *Virtual Machine Monitor*, amely a hardveren fut és úgy biztosítja a multiprogramozást, hogy több virtuális gépet emulál a felette lévő réteg számára.

- Az *emulált gépek* nemcsak *kiterjesztett gép* másolatuk a *valódi gépeknek*, beleértve a *kódmótváltási mechanizmust*, az *I/O eszközeiket* a *rendszer*ben.
- Szóval mindent tud, ami egy valódi gépen van.

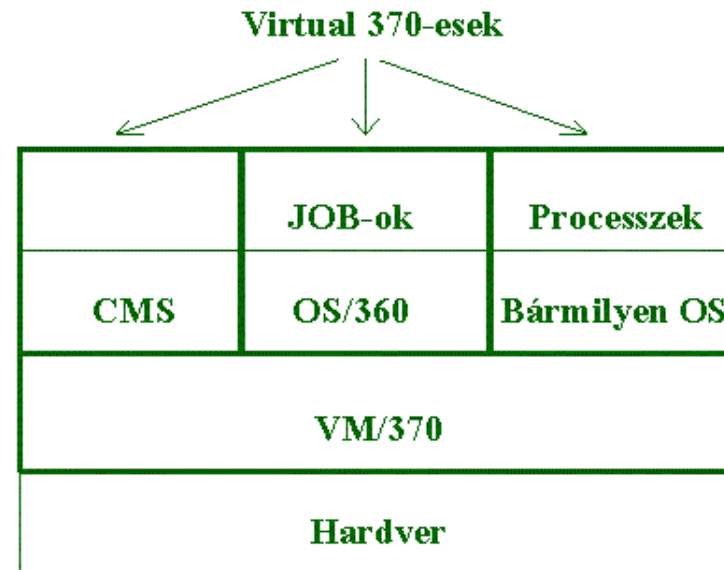


A VM/370 rendszer

Virtuális gép

- *Időkiosztásos rendszere* biztosítja a *multiprogramozást* (több virtuális gépet emulál a felette lévő réteg számára),

Pl.: VM/370, CMS,
a MACH mikrokernél

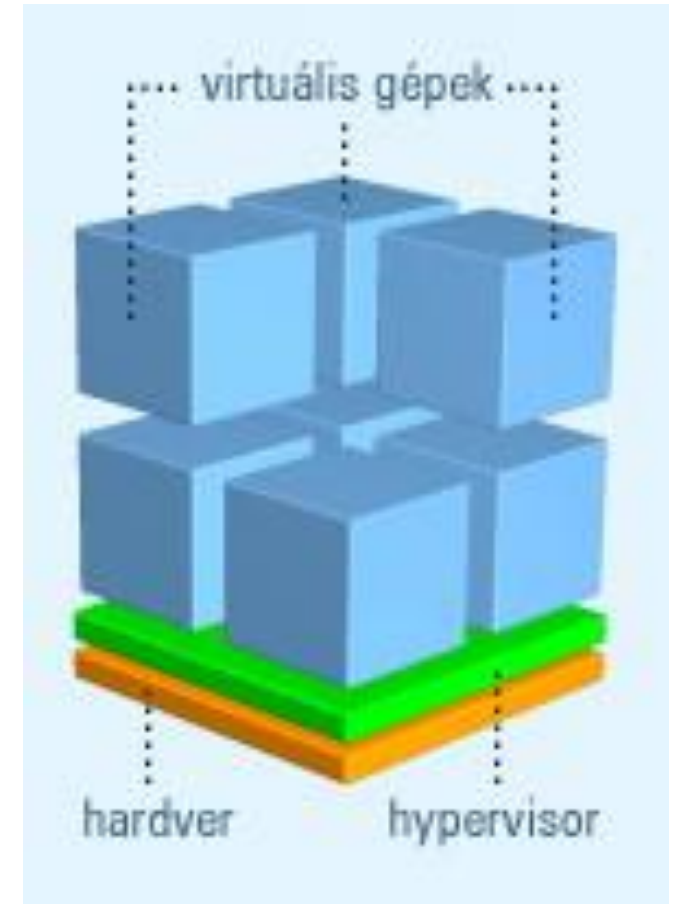


Virtuális gép

Virtuális gépek

A Virtual Machine Monitor (VMM) fut a puszta hardveren fut.

- Az emulált gépeken különböző operációs rendszerek futtathatók.
- A VM a hardver feletti réteg közvetlenül.

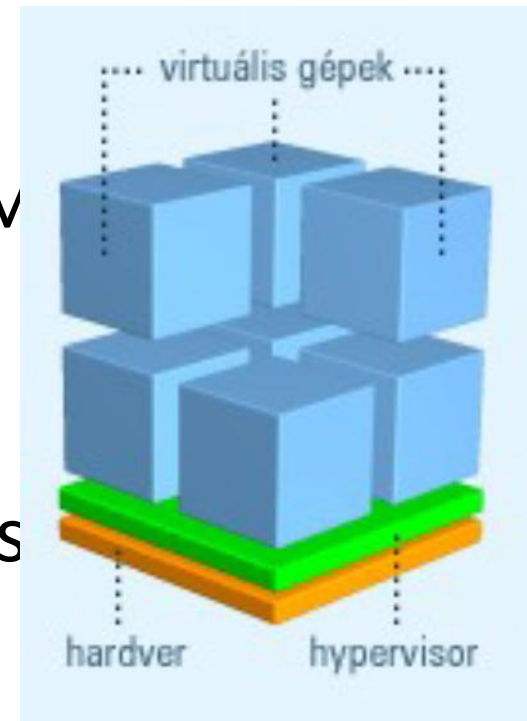


Virtuális gép (VM)

A *virtuális gép* tulajdonképpen egy *valódi számítógép emulációját (másolatát)* takarja.

A VM a gazdaszámítógép erőforrásait (RAM, CPU, stb.) használja fel, így ha több VM-et futtatunk egyszerre, az erőforrásokat osztozniuk kell.

Ezt az ún. *hypervisor* réteg segíti, ez biztosítja a virtuális gépek közötti kommunikációt a fizikai számítógép operációs rendszere és a VM-ek között.



Virtuális gép (VM)

Virtuális gép *tartalmazza a teljes szoftver-, és hardver stack-et*, amin az *alkalmazások* futnak azaz:

- a *virtualizált hardvereket* (hálózati adapter, CPU, merevlemezek, stb.),
- a teljes *operációs rendszert*, és a
- *felhasználói területet*.

Docker - kitérő

„Aki, ma a szoftverfejlesztő szakmában dolgozik érdemes megismerkedni a témával.

- Ez egy hasznos eszköz az *applikációk containerekbe csomagolására*, és abban történő futtatására.
- Különbség a *container* és a *VM* között az, hogy a *container*ek csak a *felhasználói teret virtualizálják*, tehát a *többi erőforrás közös*.

Docker – container - kitérő

Tehát, a *hardver* és az *OS* nem része a “csomagnak”, csak alkalmazásunkhoz kötődő *könyvtárak*, *bináris* és a *felhasználó terület*.

Docker jellemzők - kitérő

A Docker az egyik legelterjedtebb *container framework*, amelynek implementációja támaszkodik a Linux kernel nyújtotta lehetőségekre.

- Könnyű használhatóság,
- Sebesség,
- Modularitás és a
- Skálázhatóság.

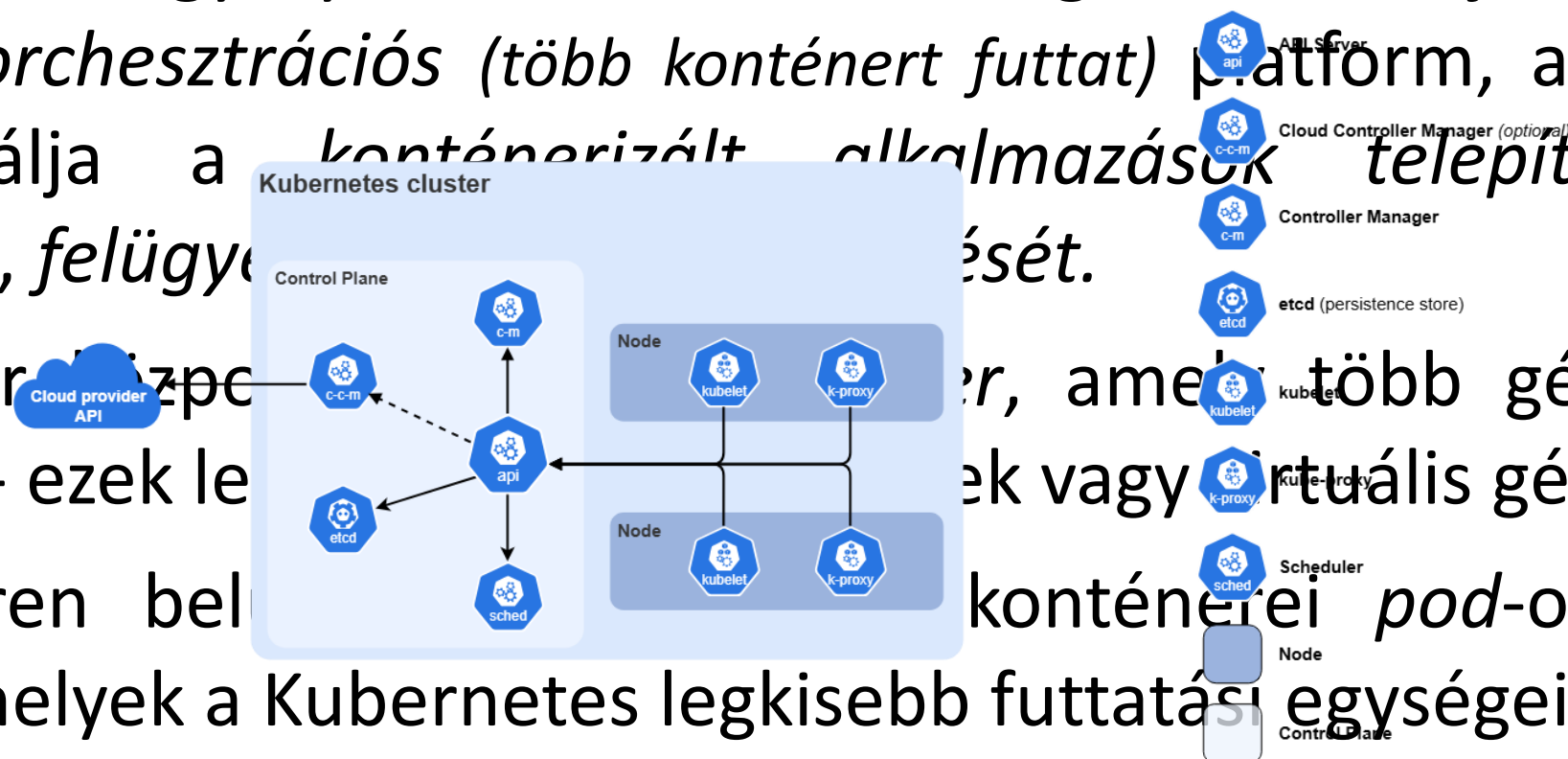
További részletek: <https://www.docker.com/>

Kubernetes környezet - kitérő

A *Kubernetes* egy nyílt forráskódú, a Google által kifejlesztett *konténer-orchesztrációs* (több konténert futtat) platform, amely automatizálja a *konténerizált alkalmazások* telepítését, skálázását, felügyeletét.

A rendszer *széles körben* (node) áll - ezek lehetnek fizikai vagy virtuális gépek.

A klaszteren belül futnak, amelyek a Kubernetes legkisebb futtatási egységei.



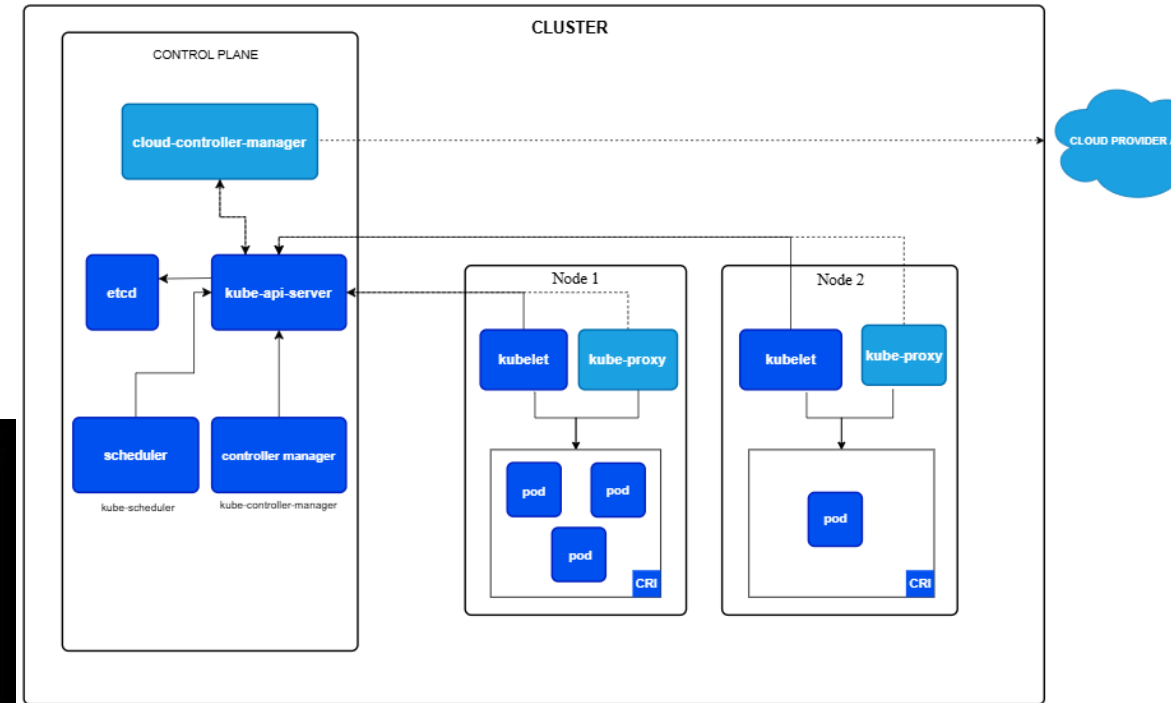
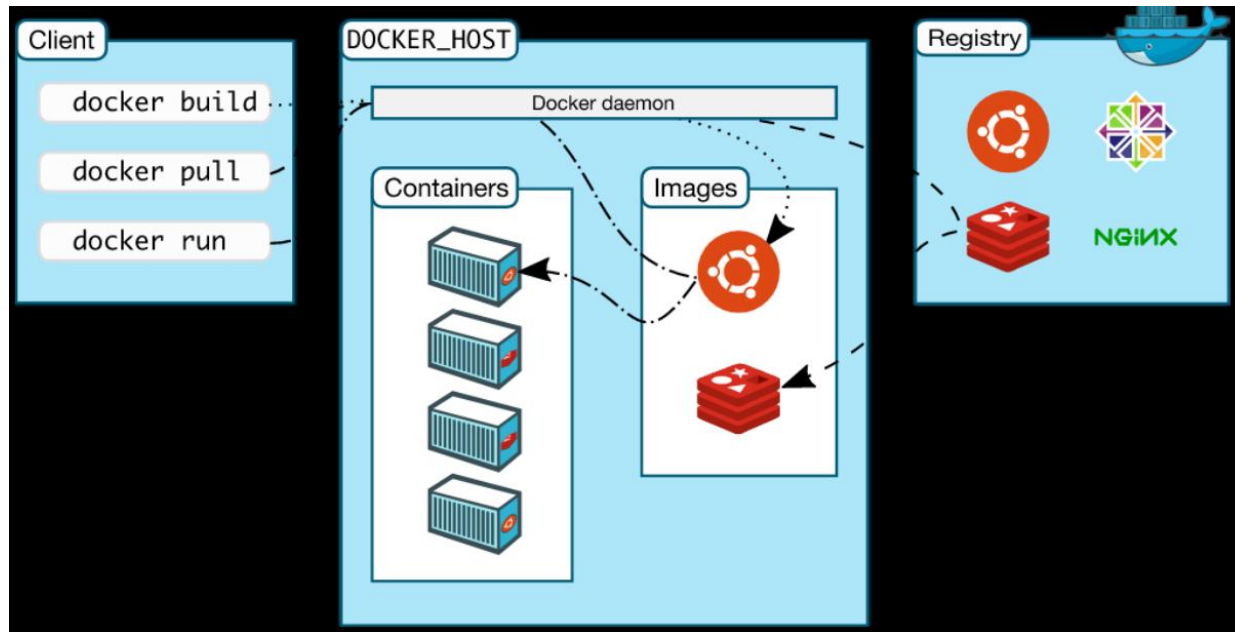
Kubernetes környezet - kitérő

Futtatható felhőben és saját infrastruktúráján.

Használják:

- Mikroservice architektúra.
- CI/CD pipeline - Continuous Integration/ Deployment
- Skálázható webalkalmazások.
- Big Data / AI workload.
- DevOps környezet.

Kubernetes vs. Docker



<https://www.docker.com/>

Skálázás
Terheléselosztás
Komplexitás

Korlátozott
✗
Egyszerűbb

Automatikus
✓
Összetettebb

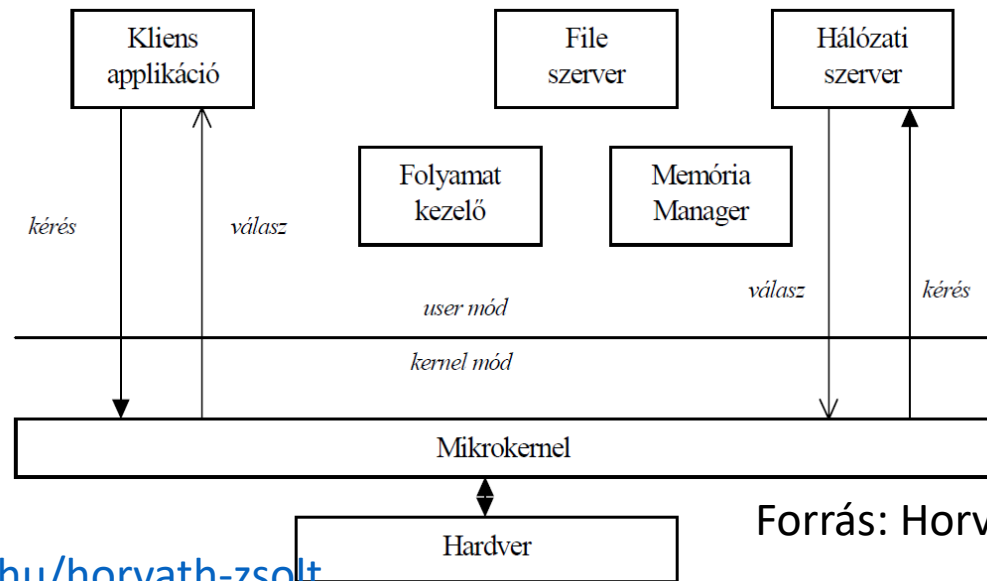
Kliens-szerver modell - mikrokernel

Igény a kisebb kernelre: *mikrokernel*

- *Lényege: felhasználói* (kliens) processz-ek szolgáltatást igényeljenek a *szolgáltató* processzekről.
- *A szolgáltató processzek* programjai *önállóan linkelhetők*.
- Betölthetnek a *rendszerbe teljesen*, vagy *időlegesen*.
(daemon processzek - olyan feladatok, amiket a háttérben (gyakran "észrevétlenül"))

Kliens-szerver modell

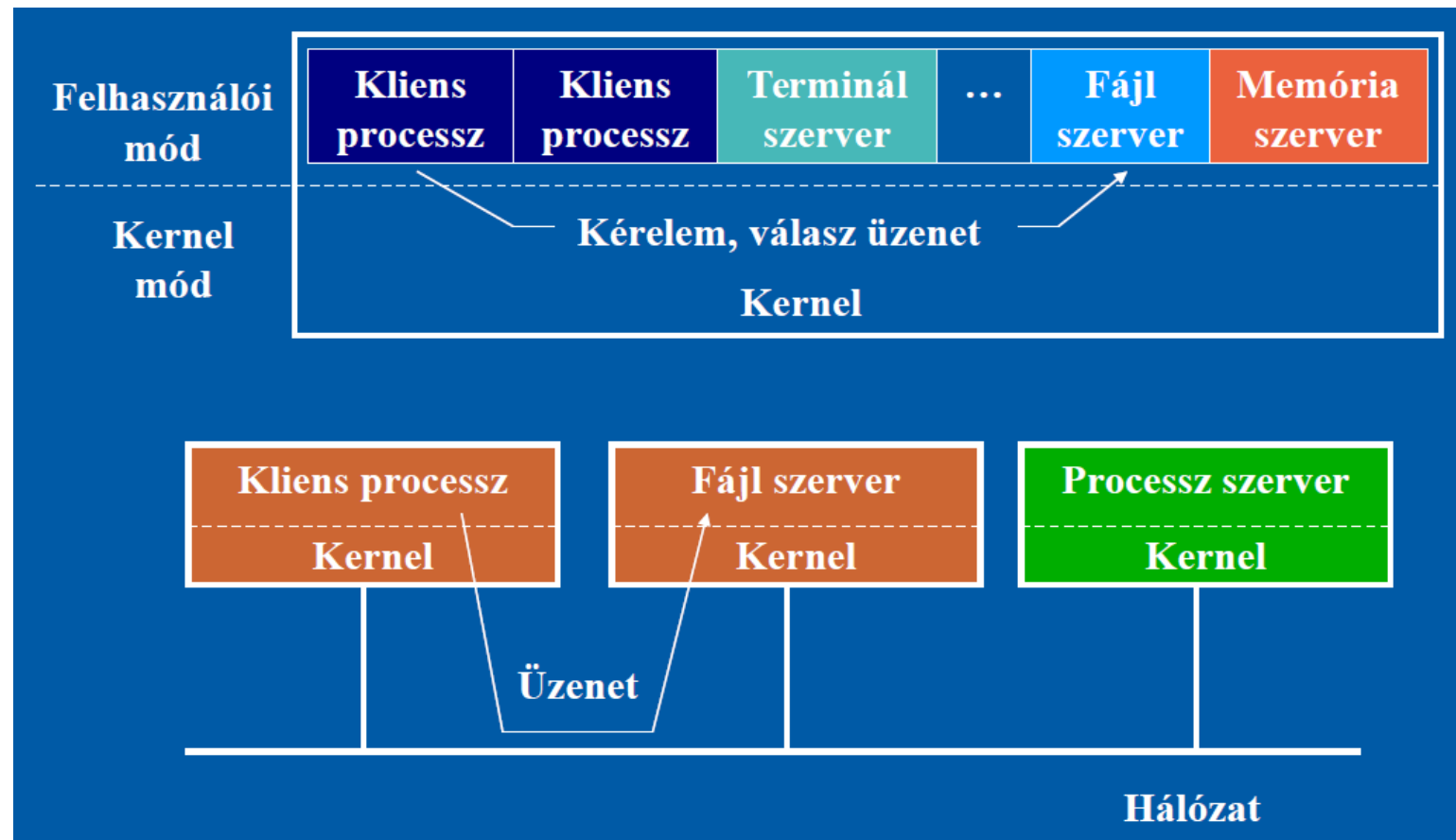
- Futhatnak *kernel-*, vagy *user módban*.
- Processzek közti kommunikációval *kérhetők a szolgáltatások*.
- Ebből fejlődtek ki a *hálózati osztott rendszerek*.



Forrás: HorvathZS

Kliens-szerver modell

Kliens-szerver modell

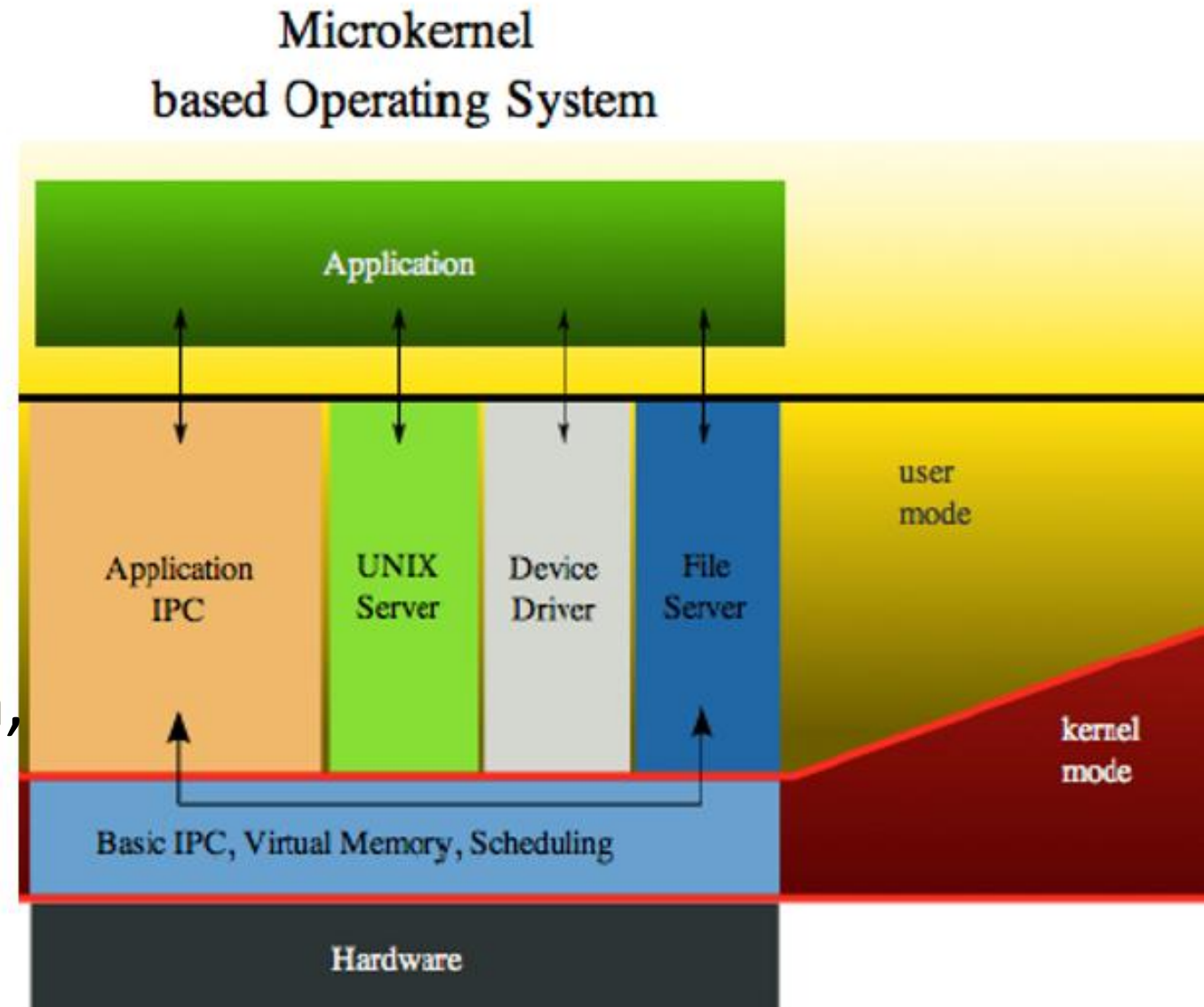


Kliens-szerver modell

Forrás: Horányi, Micskei

Microkernel összetevői:

- Virtuális memória - a Microkernelbe.
- Processz ütemezés.
- Inter-Process Communication, processzek közötti kommunikáció.



Kliens-szerver modell - előnyei

- A mikrokernel architektúra *kicsi és elszigetelt*, ezért jobban tud működni.
- A mikrokernelek *biztonságosak*.
- A rendszer *bővítése könnyebben* elérhető.
- A mikrokernelek *modulárisak*, a különböző modulok a kernel érintése nélkül cserélhetők, újratölthetők, módosíthatók.
- *Kevesebb meghibásodás* a monolit rendszerekhez képest.

Kliens-szerver modell - hátrányai

- A szolgáltatások nyújtása *mikrokernel rendszerben költséges* a normál monolitikus rendszerhez képest.
- *Kontextusváltásra* van szükség, ha az illesztőprogramokat *eljárásként* vagy *folyamatként* implementálják.

Kliens-szerver modell

- Önálló, jól definiált *funkciókat ellátó komponensek*.
- Mag az ún. *mikrokernel* (mikrokernel-es operációs rendszerek).

Funkciója:

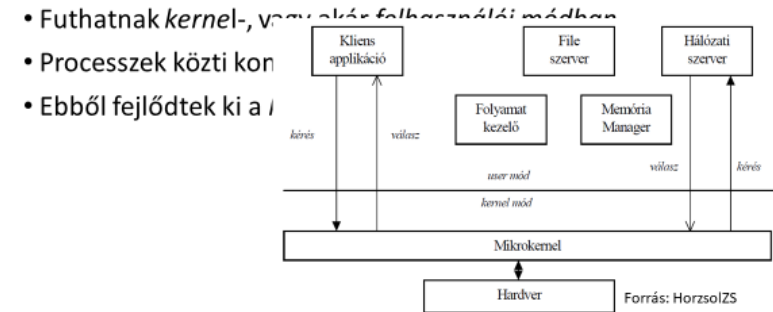
- Szolgáltatás *kéréseket üzenetek* formájában továbbítja az operációs rendszer moduljai számára.
- Biztosítja a processzek számára a hardver elérését. (Hardver elérés csak mikrokernel-en keresztül.)

Kliens-szerver modell

A modulok önálló szerver folyamatok.

- *Mikrokernel*: privilegizált (kernel) mód.
- *Folyamatok*: felhasználói mód.
- *Előny*: rugalmas, biztonságos működés.
- *Hátrány*: nem hatékony hardver kihasználás (környezetváltások!).

Kliens-szerver modell



Forrás: HorváthZS

Kliens-szerver modell

Kommunikáció: üzenet adás/vétel.

- *Szerver processz* - meghatározott szolgáltatásokat végez: *terminál-, web-, file i/o* - szolgáltatás.
- *Kliens processz*: egy szerver processz szolgáltatásait veszi igénybe,
 - *kérés* = üzenet (tartalma specifikálja a pontos kérést):
 - *válasz* = üzenet.

Kliens-szerver modell

Üzenet = fejléc + tartalom

- *Header* - az üzenet továbbításához szükséges információk: címzett (proc), feladó, hossz, kódolás, idő-bélyegzők, ...
- *Tartalom* - feladó állítja össze, címzett értelmezi.

OS feladata: az üzenetek közvetítése processzek között.

Irodalom

- Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

- Linux Mint kézikönyv

[https://linuxmint.hu/sites/default/files/fajlok/linux mint kezikonyv.pdf](https://linuxmint.hu/sites/default/files/fajlok/linux_mint_kezikonyv.pdf)

- Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.